



Game Playing: Adversarial Search

UNIT III





Adversarial Search

We have experience in search where we assume that we are the only intelligent being and we have explicit control over the “world”.

Lets consider what happens when we relax those assumptions.





Two Player Games

Max always moves first.

Min is the opponent.

We have

- An initial state.
- A set of operators.
- A terminal test (which tells us when the game is over).
- A utility function (evaluation function).



Max Vs Min

The utility function is like the heuristic function we have seen in the past, except it evaluates a node in terms of how good it is for each player. Positive values indicate states advantageous for Max, negative values indicate states advantageous for Min.





Game Trees

- Problem spaces for typical games represented as trees.
- Root node represents the “board” configuration at which a decision must be made as to what is the **best single move to make next**. (not necessarily the initial configuration)
- **Evaluator function** rates a board position. $f(\text{board})$ (a real number).
- Arcs represent the possible legal moves for a player (no costs associates to arcs)
- Terminal nodes represent end-game configurations (the result must be one of “win”, “lose”, and “draw”, possibly with numerical payoff)



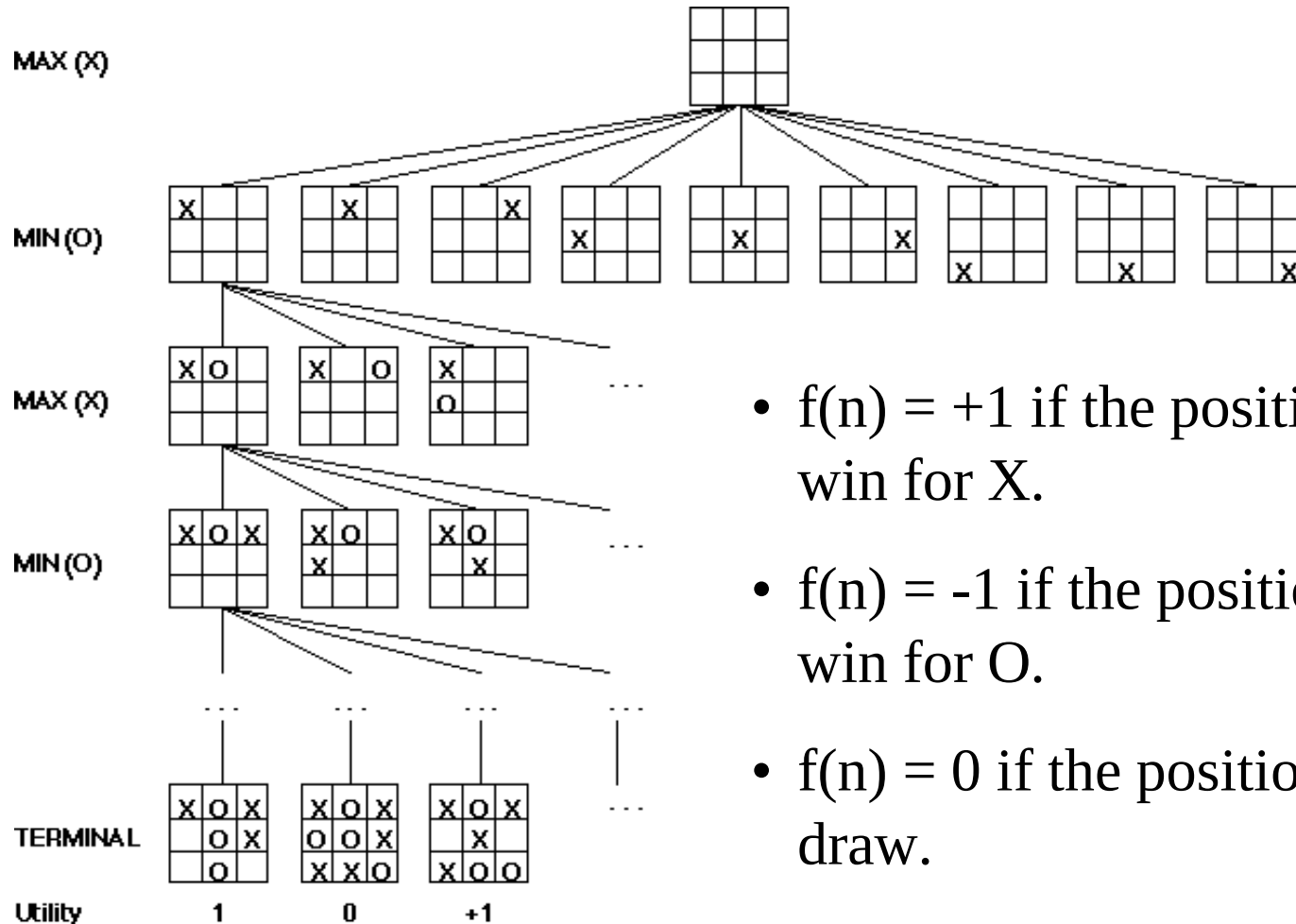


- If it is my turn to move, then the root is labeled a "**MAX**" node; otherwise it is labeled a "**MIN**" node indicating my opponent's turn.
- Each level of the tree has nodes that are all MAX or all MIN; nodes at level i are of the opposite kind from those at level $i+1$
- Complete game tree: includes all configurations that can be generated from the root by legal moves (all leaves are terminal nodes)
- **Incomplete game tree**: includes all configurations that can be generated from the root by legal moves to a given depth (looking ahead to a given steps)





An example (partial) game tree for Tic-Tac-Toe

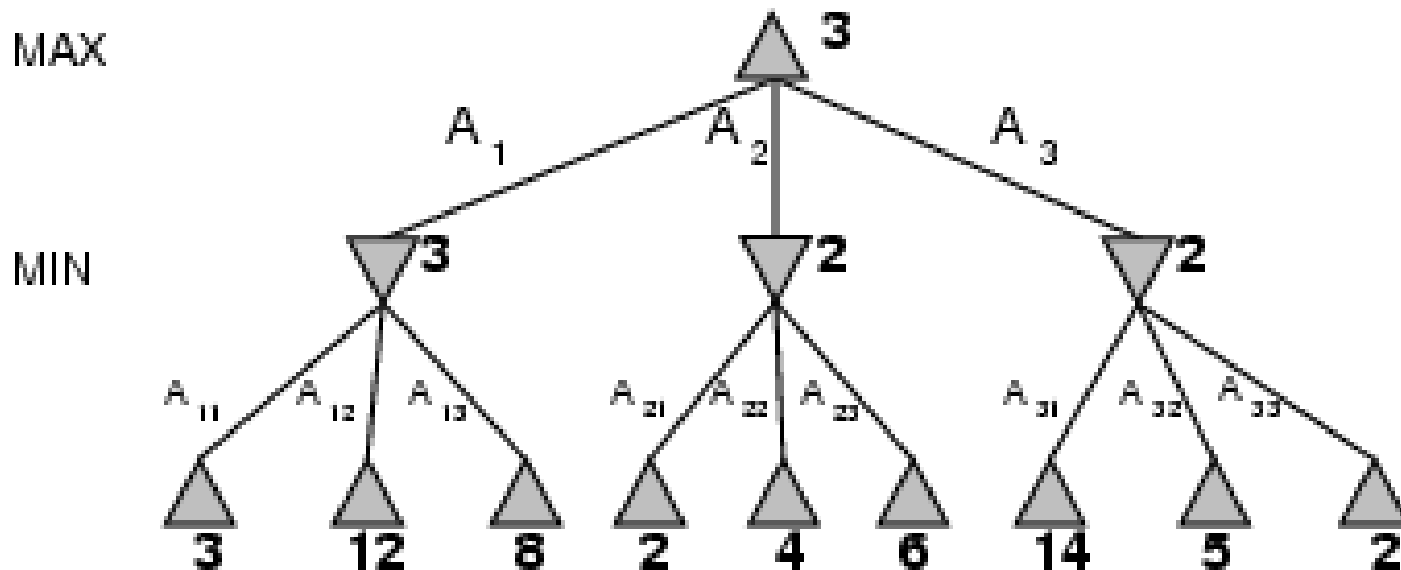


- $f(n) = +1$ if the position is a win for X.
- $f(n) = -1$ if the position is a win for O.
- $f(n) = 0$ if the position is a draw.



Minimax

- Perfect play for deterministic games
- Idea: choose move to position with highest **minimax value**
= best achievable payoff against best play
- E.g., 2-ply game:

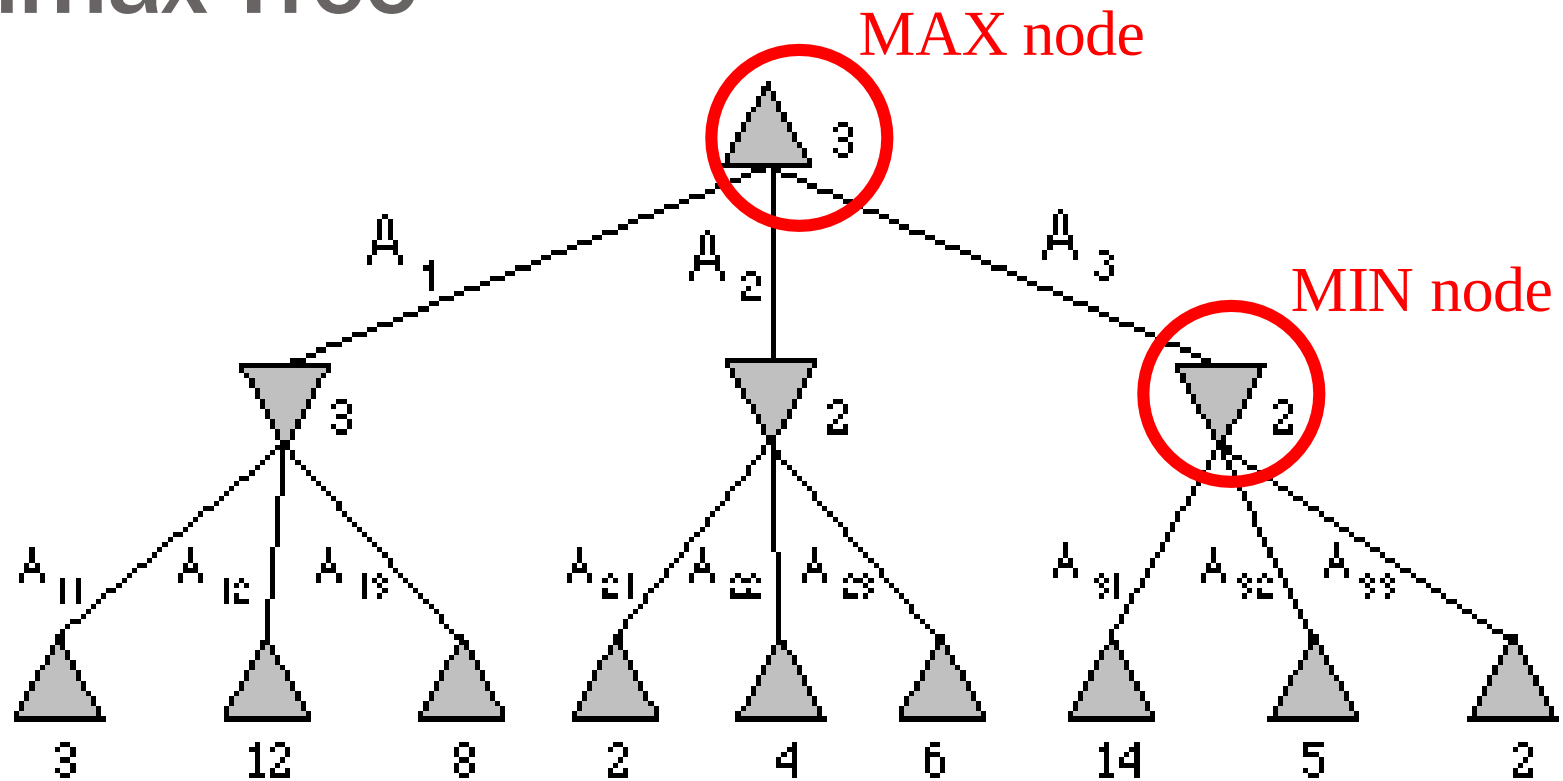




Minimax Tree

MAX

MIN



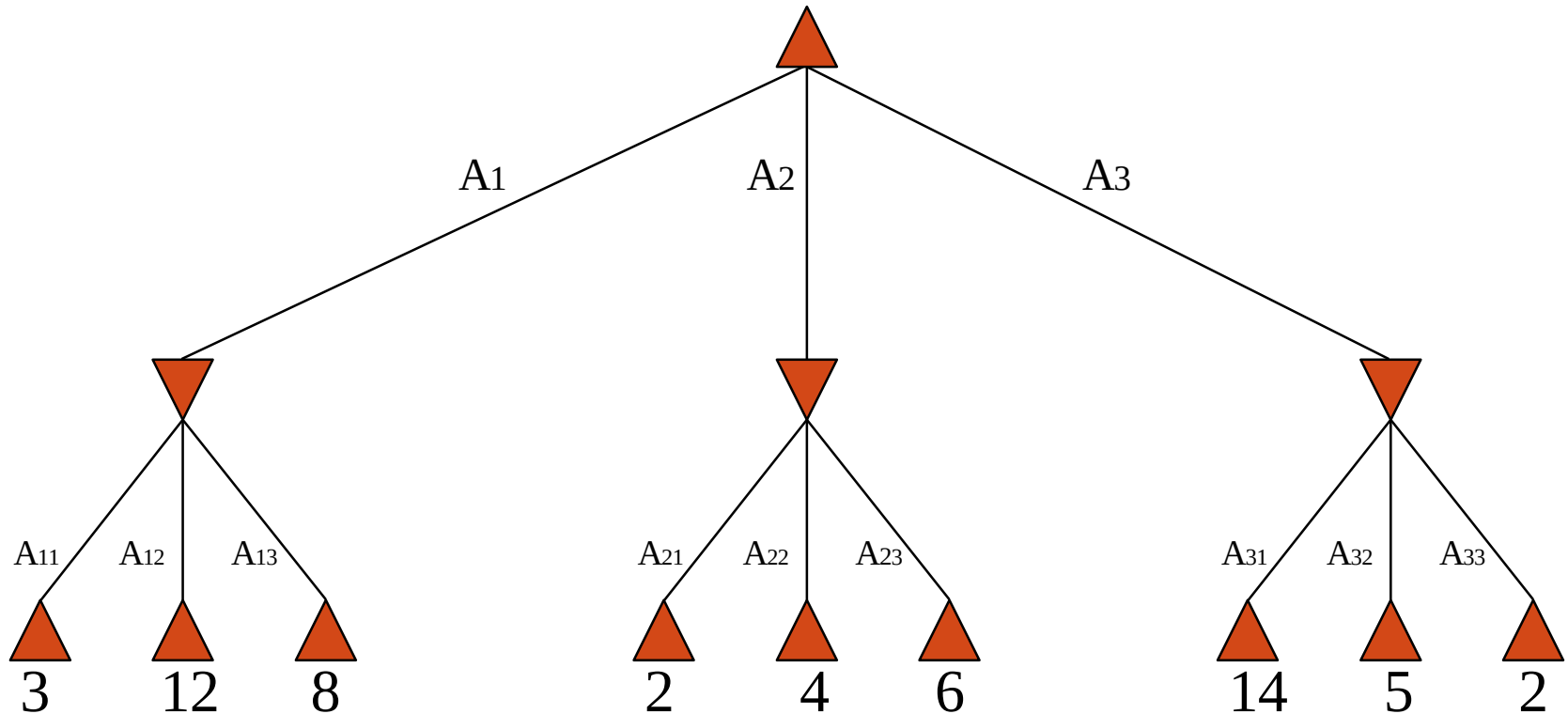
f value

A_1 is selected as the next move



A simple abstract game.

Max makes a move, then **Min** replies.



An action by one player is called a *ply*, two ply (a action and a counter action) is called a *move*.

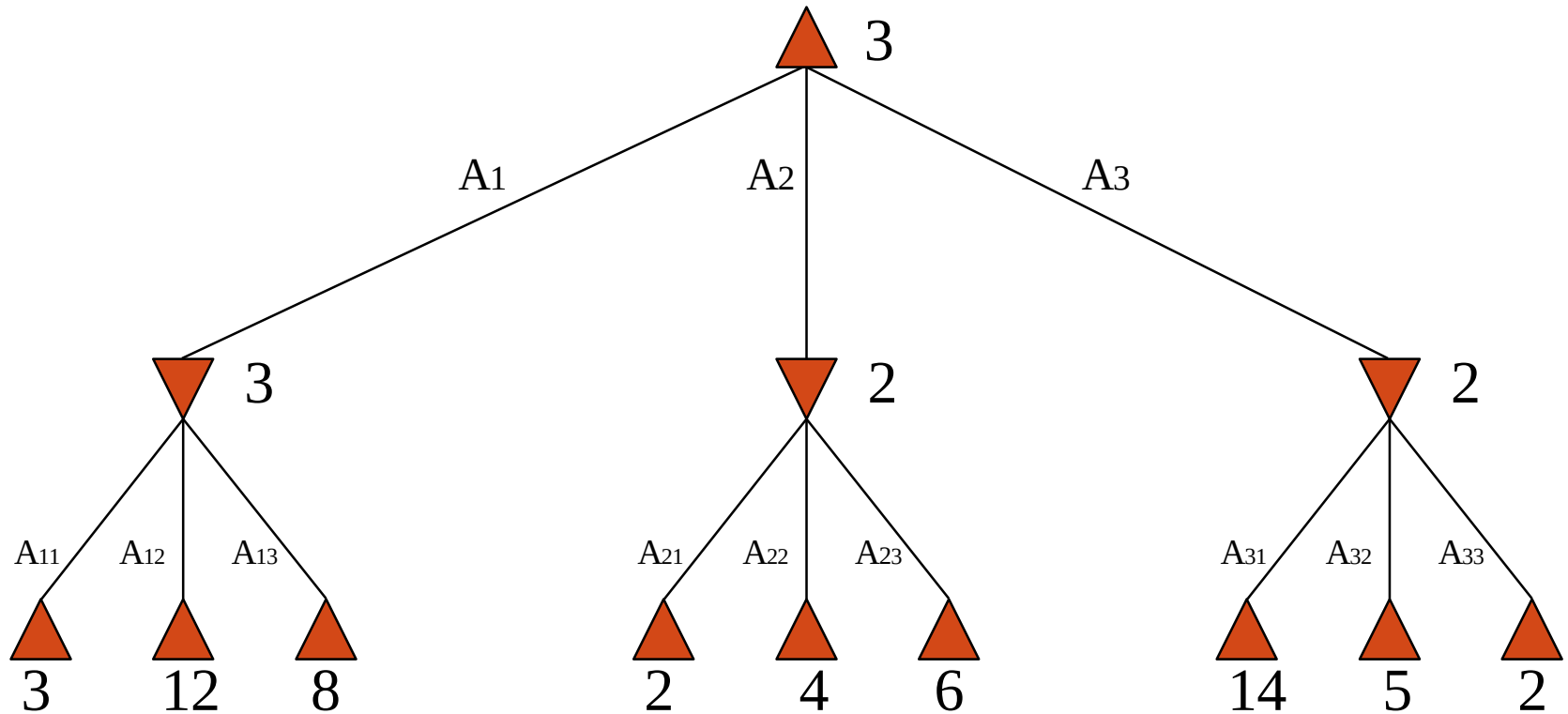




The Minimax Algorithm

- Generate the game tree down to the terminal nodes.
- Apply the utility function to the terminal nodes.
- For a S set of sibling nodes, pass up to the parent...
 - the lowest value in S if the siblings are ▲
 - the largest value in S if the siblings are ▼
- Recursively do the above, until the backed-up values reach the initial state.
- The value of the initial state is the minimum score for Max.

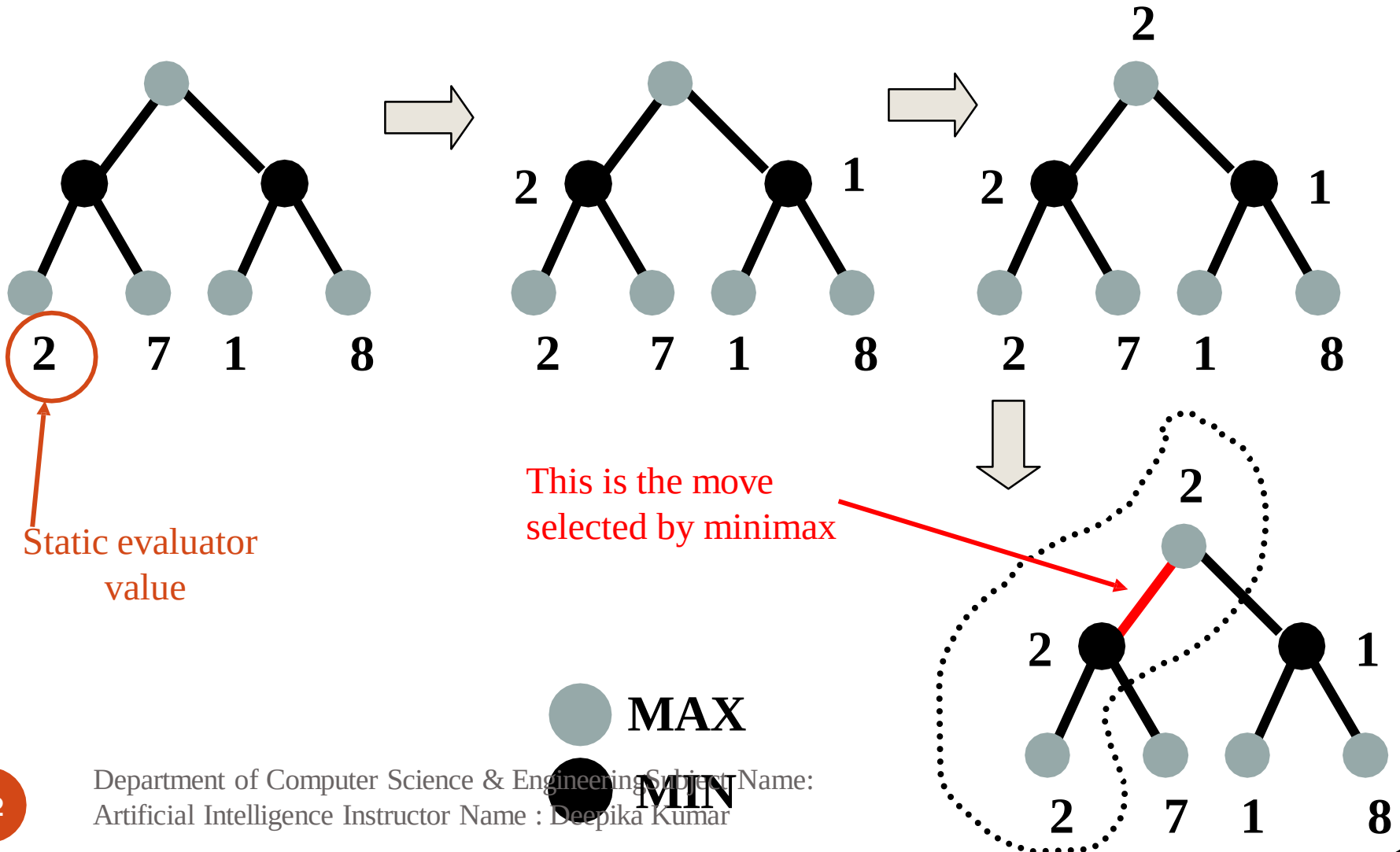




In this game Max's best move is A_1 , because he is guaranteed a score of at least 3.

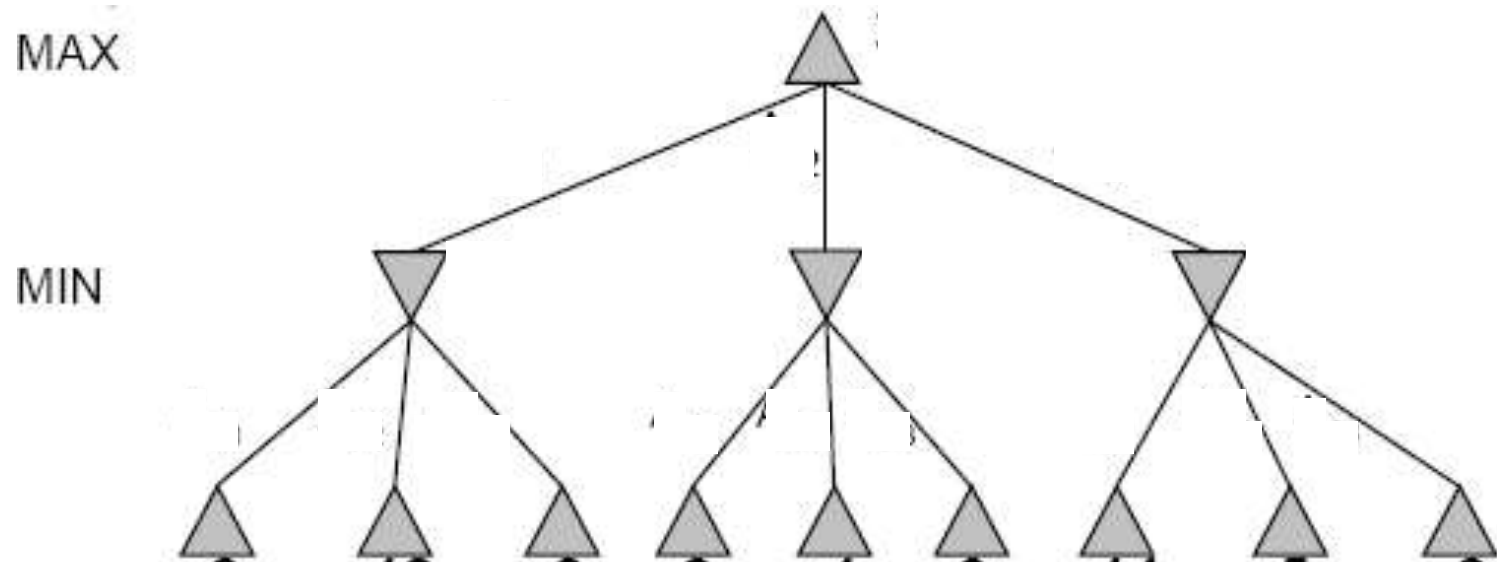


Minimax Search



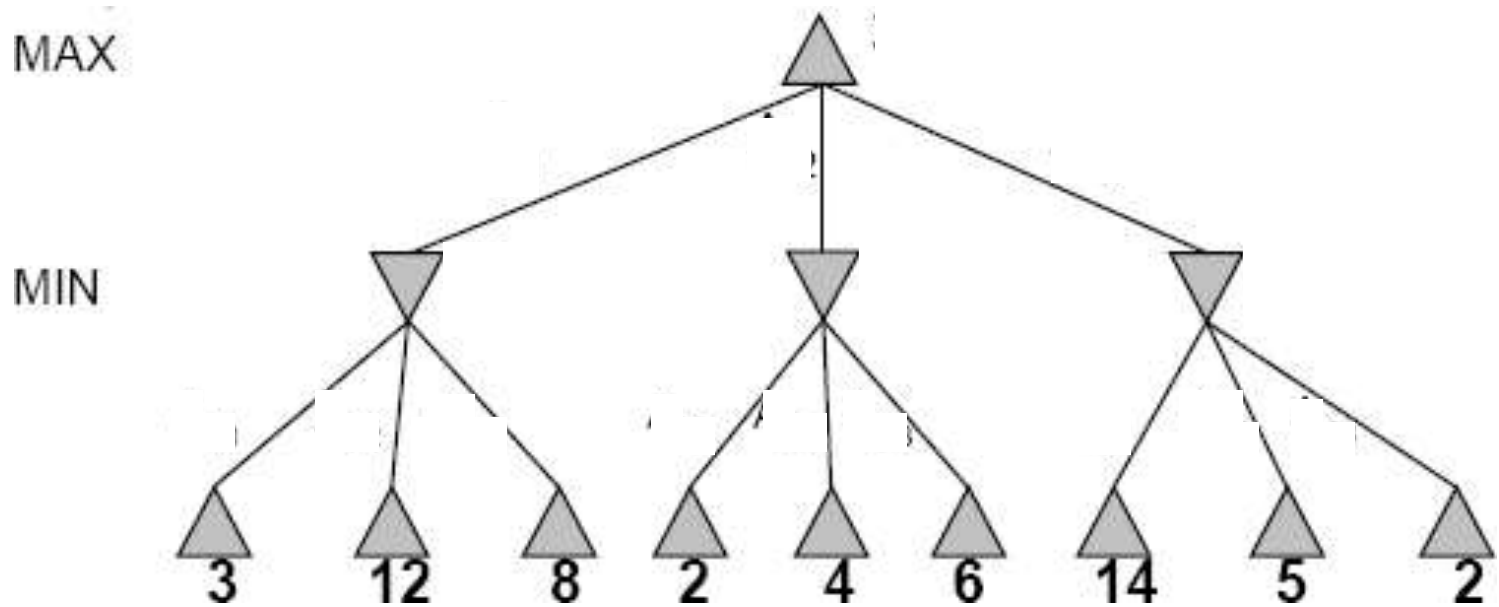


Two-Ply Game Tree



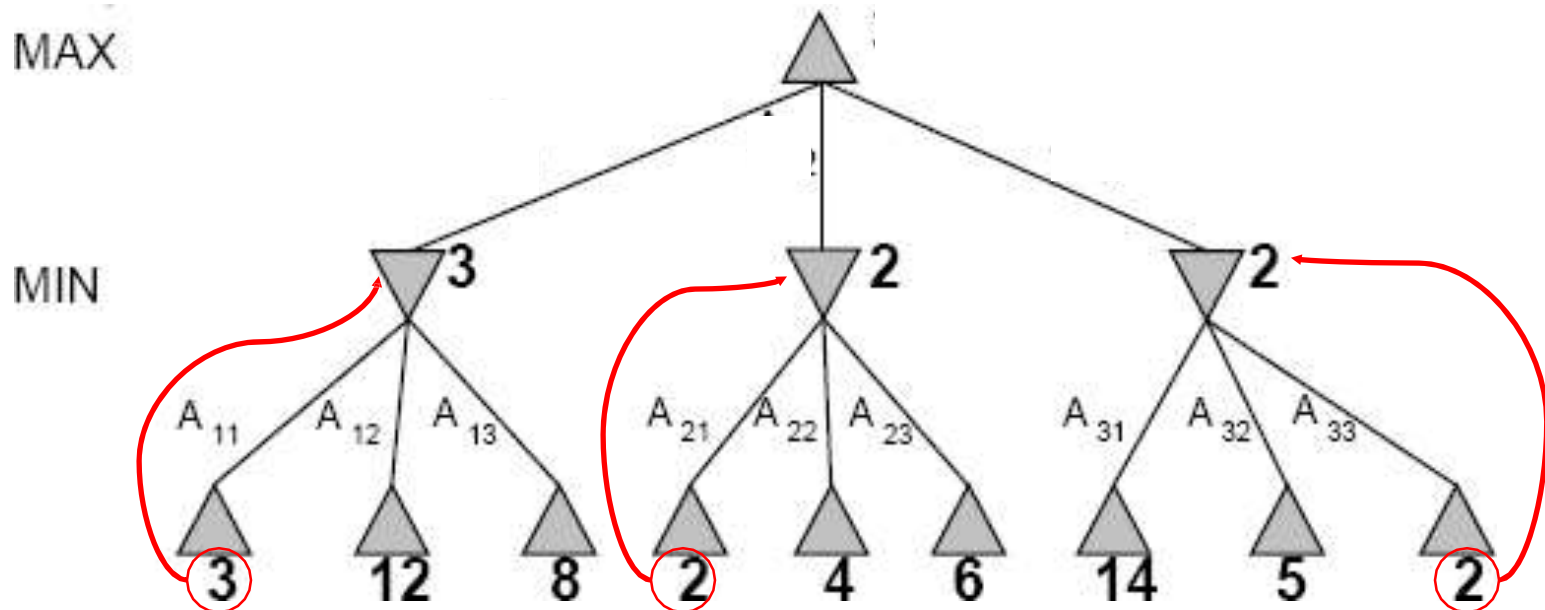


Two-Ply Game Tree





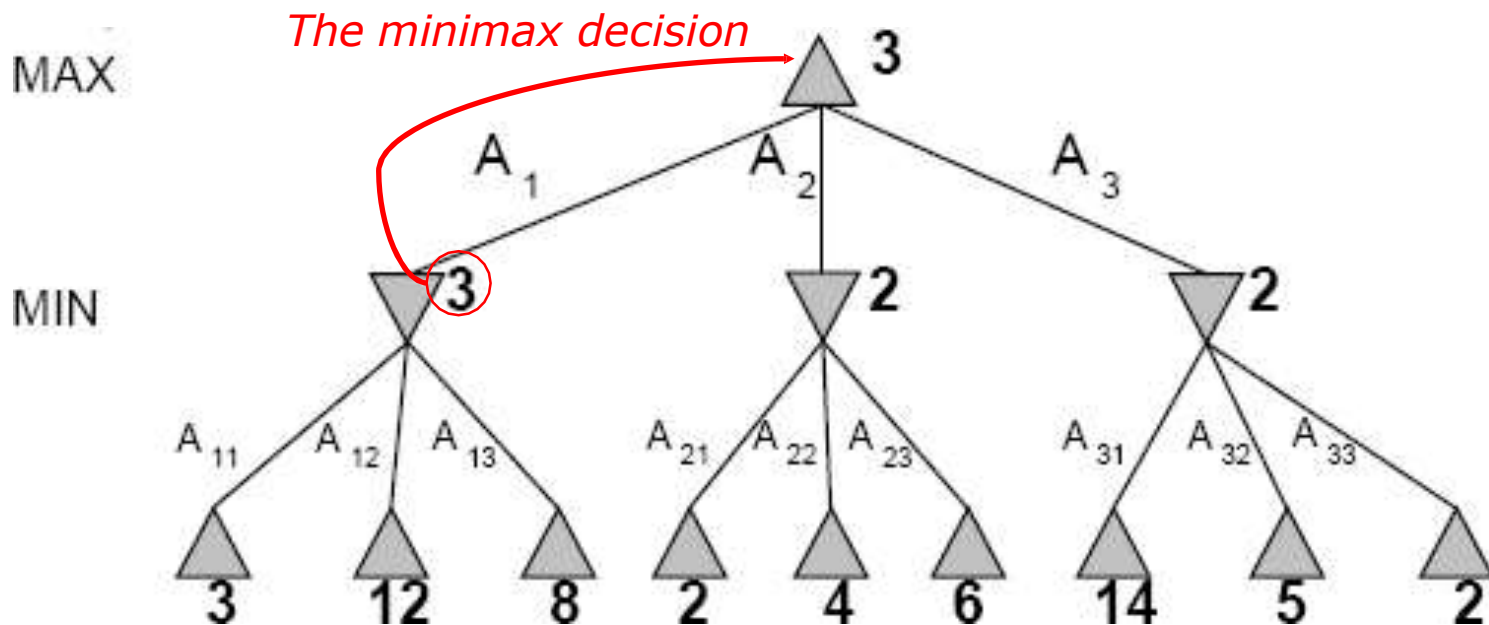
Two-Ply Game Tree





Two-Ply Game Tree

Minimax maximizes the utility for the worst-case outcome for max



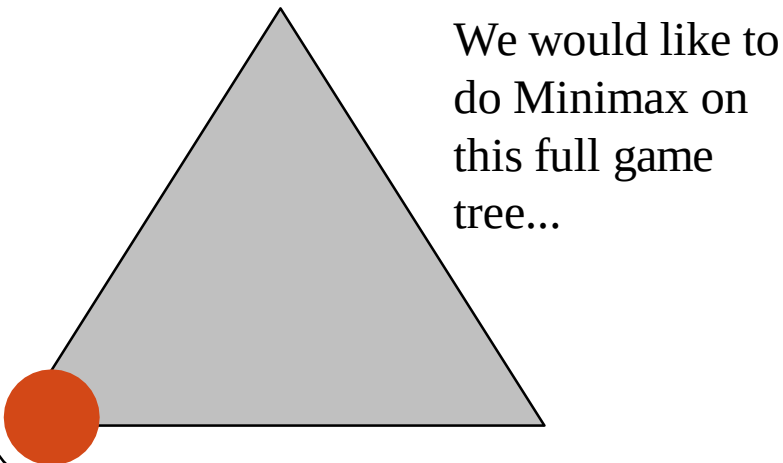


Although the Minimax algorithm is optimal, there is a problem...
The time complexity is $O(b^m)$ where b is the effective branching factor and m is the depth of the terminal states.

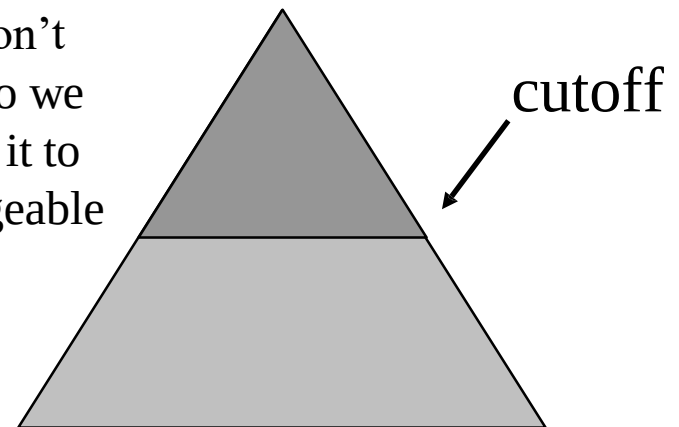
(Note space complexity is only linear in b and m , because we can do depth first search).

One possible solution is to do depth limited Minimax search.

- Search the game tree as deep as you can in the given time.
- Evaluate the fringe (border) nodes with the utility function.
- Back up the values to the root.
- Choose best move, repeat.



... but we don't have time, so we will explore it to some manageable depth.





Properties of minimax

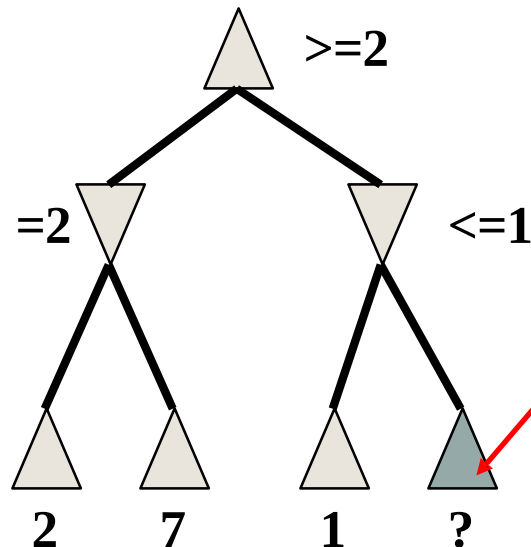
- Complete? Yes (if tree is finite)
- Optimal? Yes (against an optimal opponent)
- Time complexity? $O(b^m)$
- Space complexity? $O(bm)$ (depth-first exploration)





Alpha-beta pruning

- We can improve on the performance of the minimax algorithm through **alpha-beta pruning**.
- Basic idea: *“If you have an idea that is surely bad, don't take the time to see how truly awful it is”* -- Pat Winston



- We don't need to compute the value at this node.
- No matter what it is it can't effect the value of the root node.



Alpha-beta pruning

- Traverse the search tree in depth-first order
- At each **Max** node n , **alpha(n)** = maximum value found so far
 - Start with $-\infty$ and only increase
 - Increases if a child of n returns a value greater than the current alpha
 - Serve as a tentative lower bound of the final pay-off
- At each **Min** node n , **beta(n)** = minimum value found so far
 - Start with ∞ and only decrease
 - Decreases if a child of n returns a value less than the current beta
 - Serve as a tentative upper bound of the final pay-off.





Alpha-beta pruning

- **Alpha cutoff:** Given a Max node n , cutoff the search below n (i.e., don't generate or examine any more of n 's children) if $\alpha(n) \geq \beta(n)$
(α increases and passes β from below)
- **Beta cutoff.:** Given a Min node n , cutoff the search below n (i.e., don't generate or examine any more of n 's children) if $\beta(n) \leq \alpha(n)$
(β decreases and passes α from above)
- Carry α and β values down during search
Pruning occurs whenever $\alpha \geq \beta$

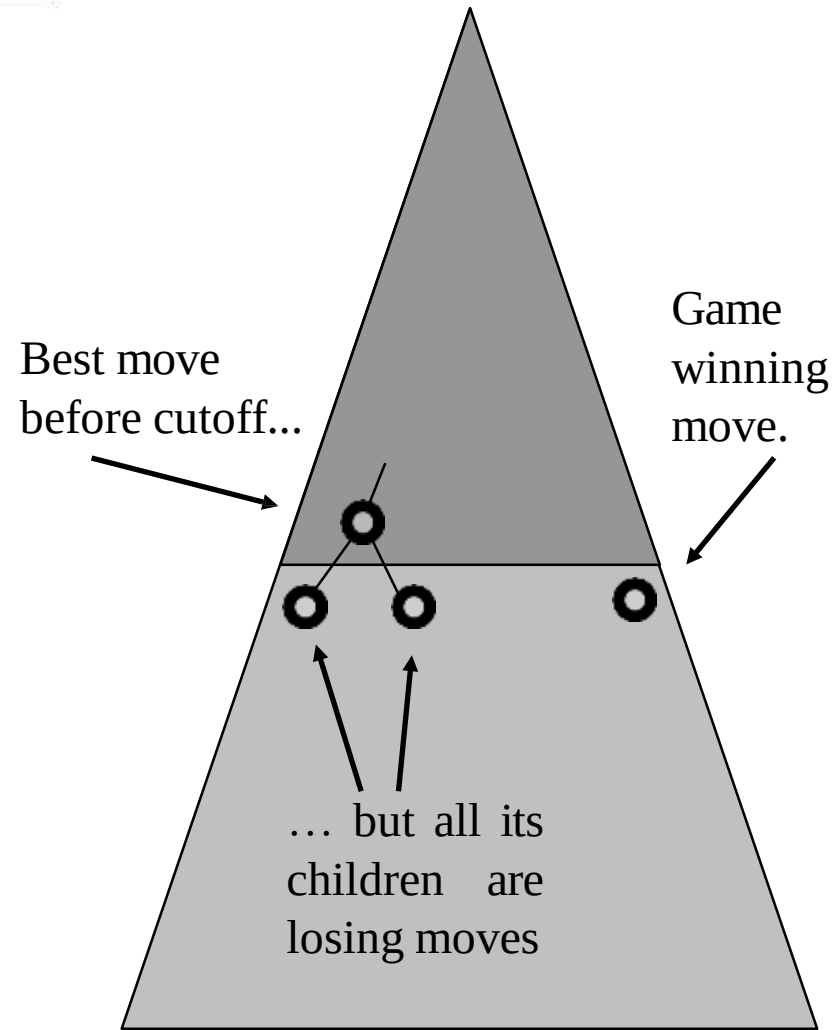


Alpha-Beta Pruning I

We have seen how to use Minimax search to play an optional game.

We have seen that because of time limitations we may have to use a cutoff depth to make the search tractable. Using a cutoff causes problems because of the “horizon” (possibility) effect.

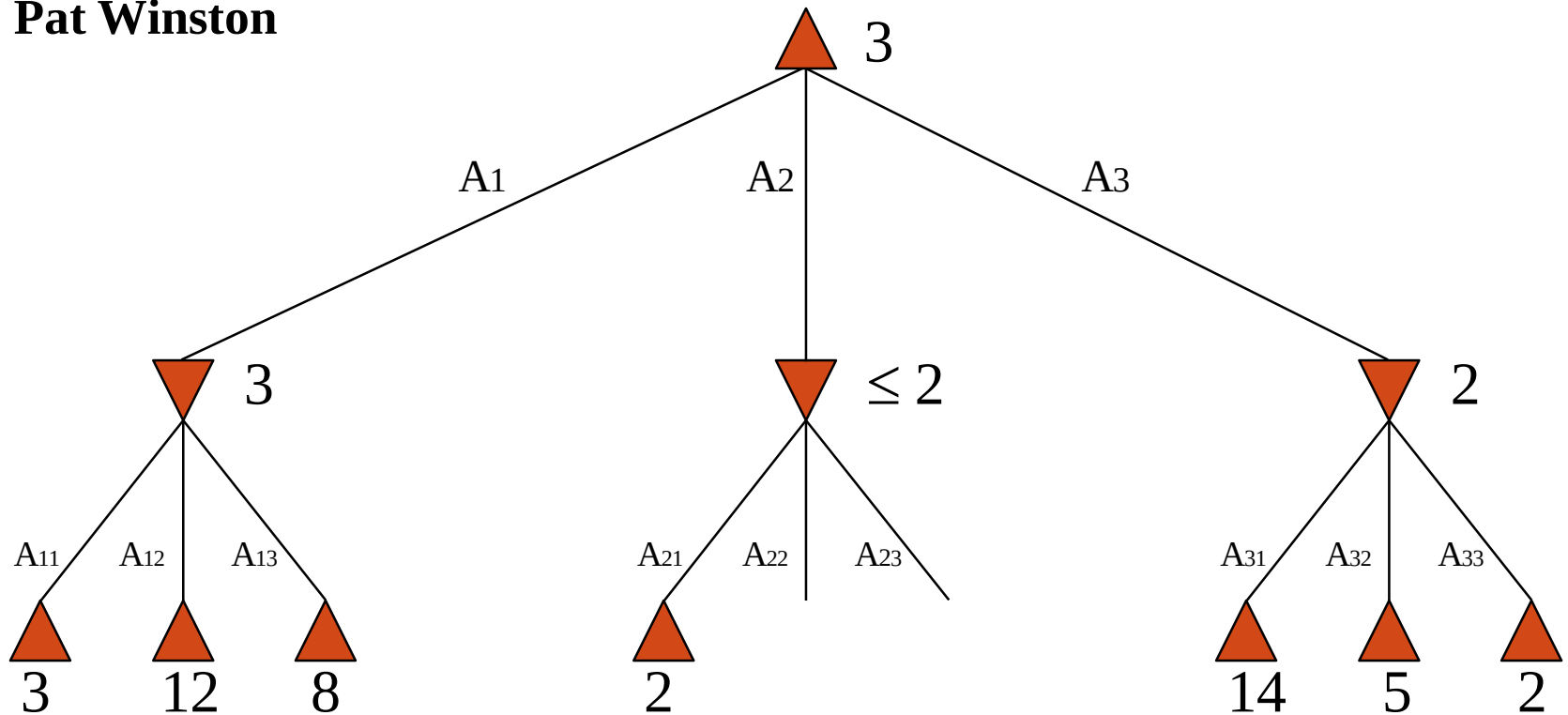
Is there some way we can search deeper in the same amount of time?
Yes! Use Alpha-Beta Pruning...





Alpha-Beta Pruning II

If you have an idea that is surely bad, don't take the time to see how truly awful it is. **Pat Winston**





Alpha-Beta Pruning III

Effectiveness of Alpha-Beta

- Alpha-Beta is guaranteed to compute the same Minimax value for the root node as computed by Minimax
- In the worst case Alpha-Beta does NO pruning, examining b^d leaf nodes, where each node has b children and a d -ply search is performed
- In the best case, Alpha-Beta will examine only $2b^{d/2}$ leaf nodes. Hence if you hold fixed the number of leaf nodes then you can search twice as deep as Minimax!
- The best case occurs when each player's best move is the leftmost alternative (i.e., the first child generated). So, at MAX nodes the child with the largest value is generated first, and at MIN nodes the child with the smallest value is generated first. This suggest that we should order the operators carefully...

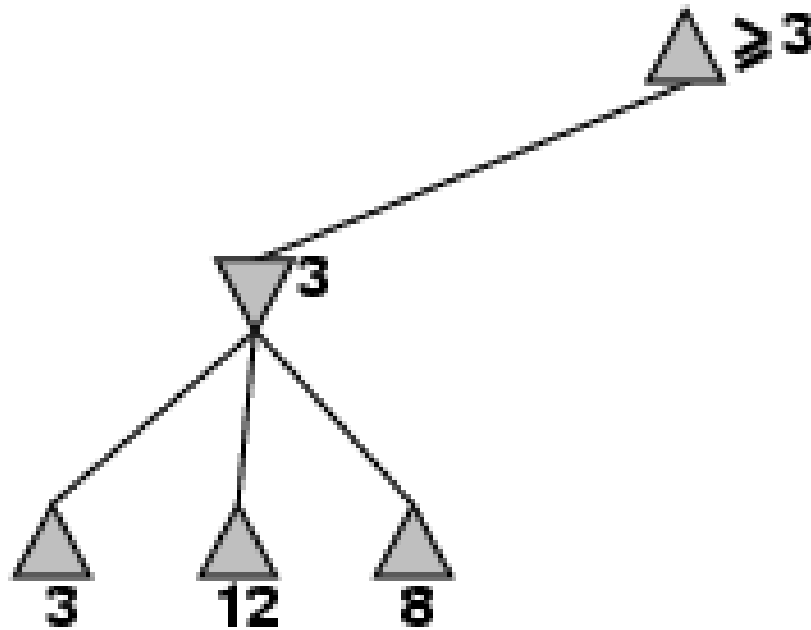
In the chess program Deep Blue, they found empirically that Alpha-Beta pruning meant that the average branching factor at each node was about 6 instead of about 35-40



α - β pruning example

MAX

MIN

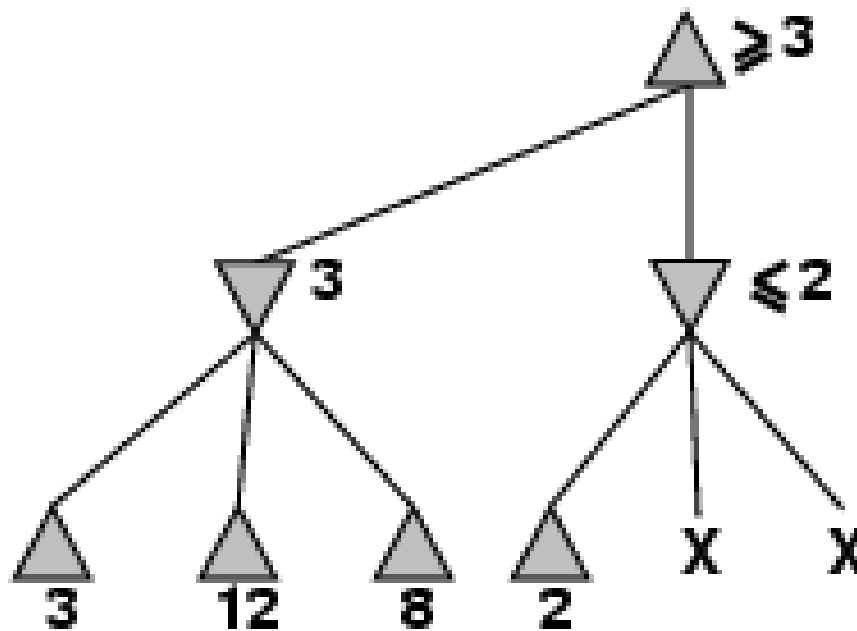




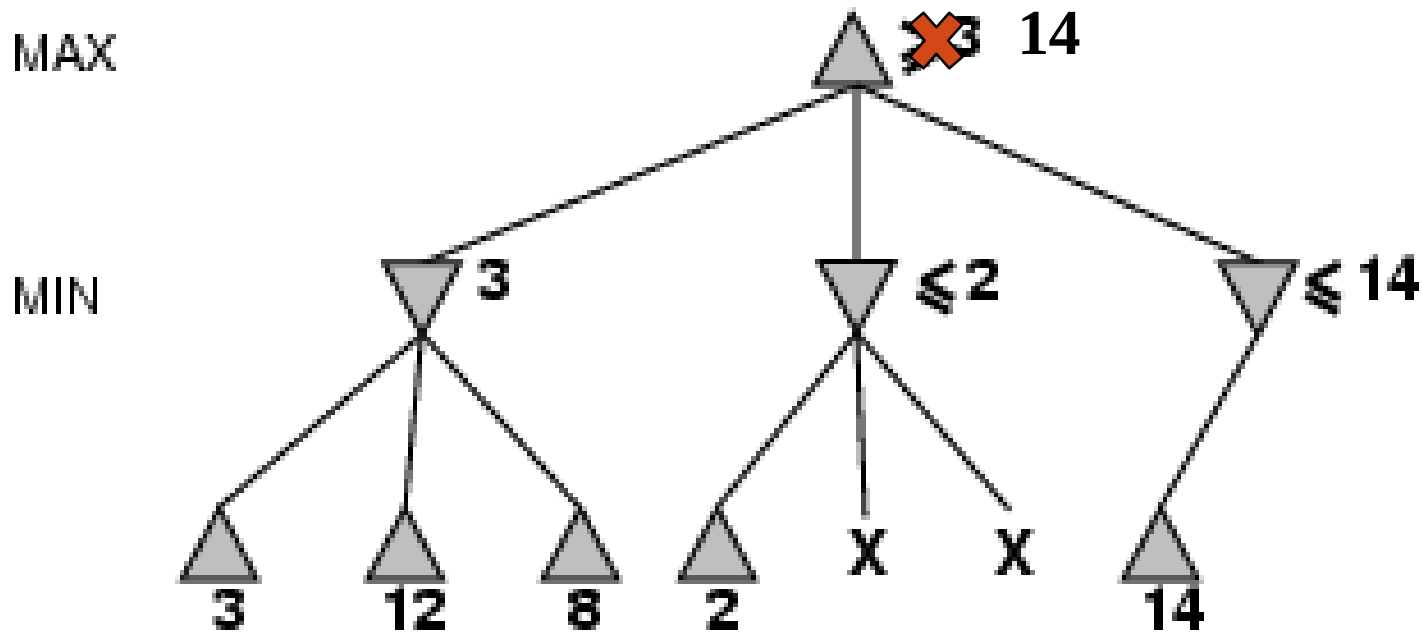
α - β pruning example

MAX

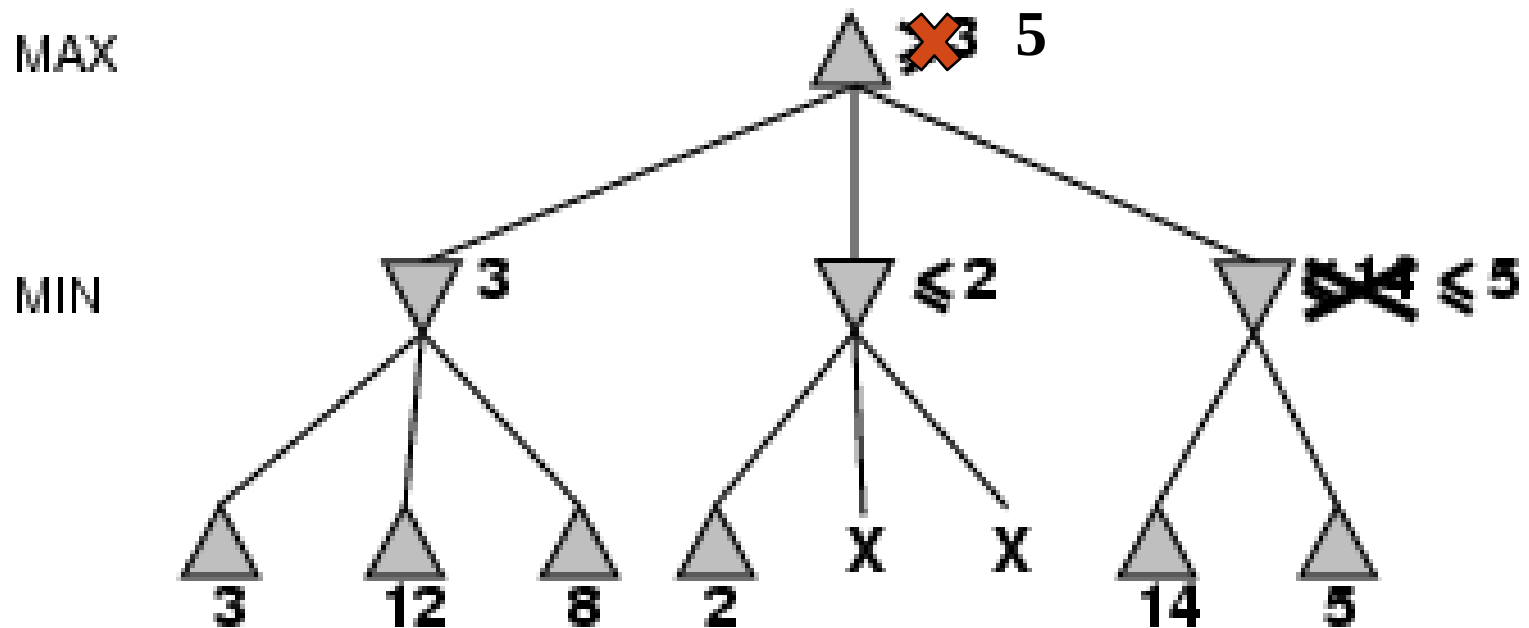
MIN



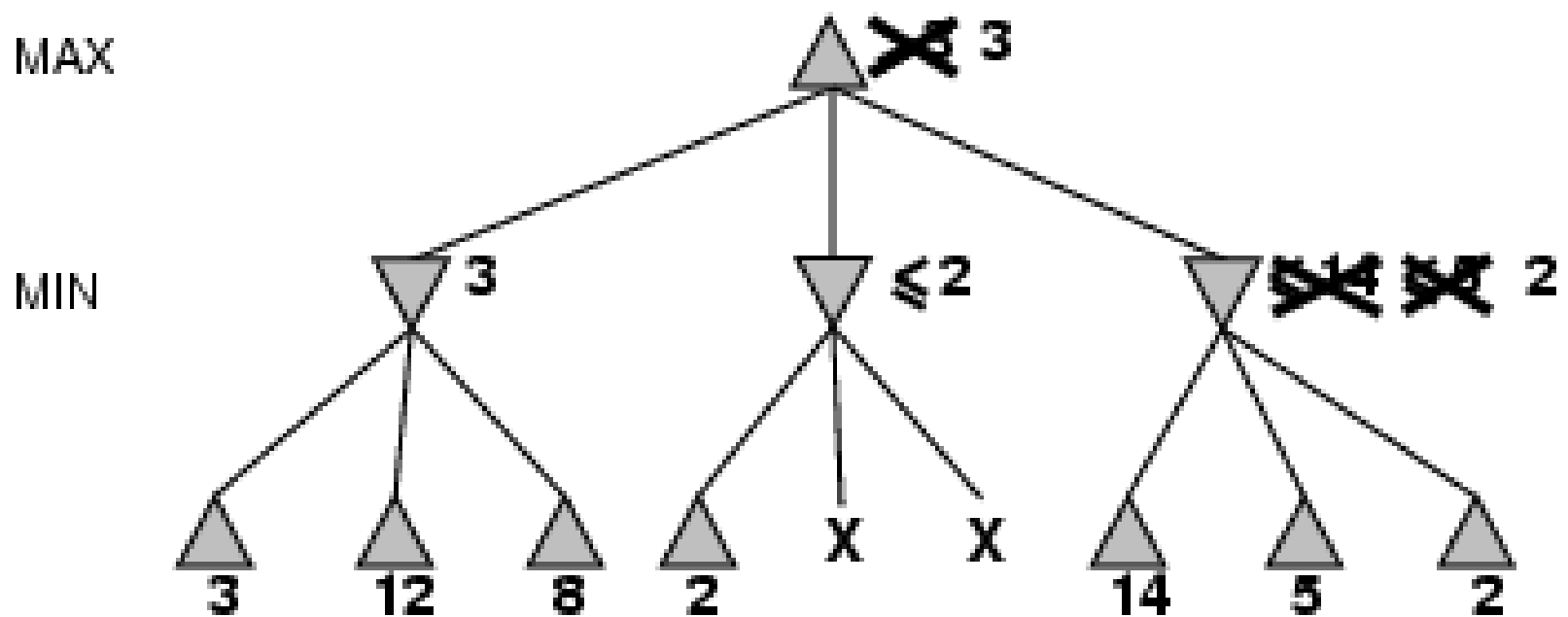
α - β pruning example



α - β pruning example

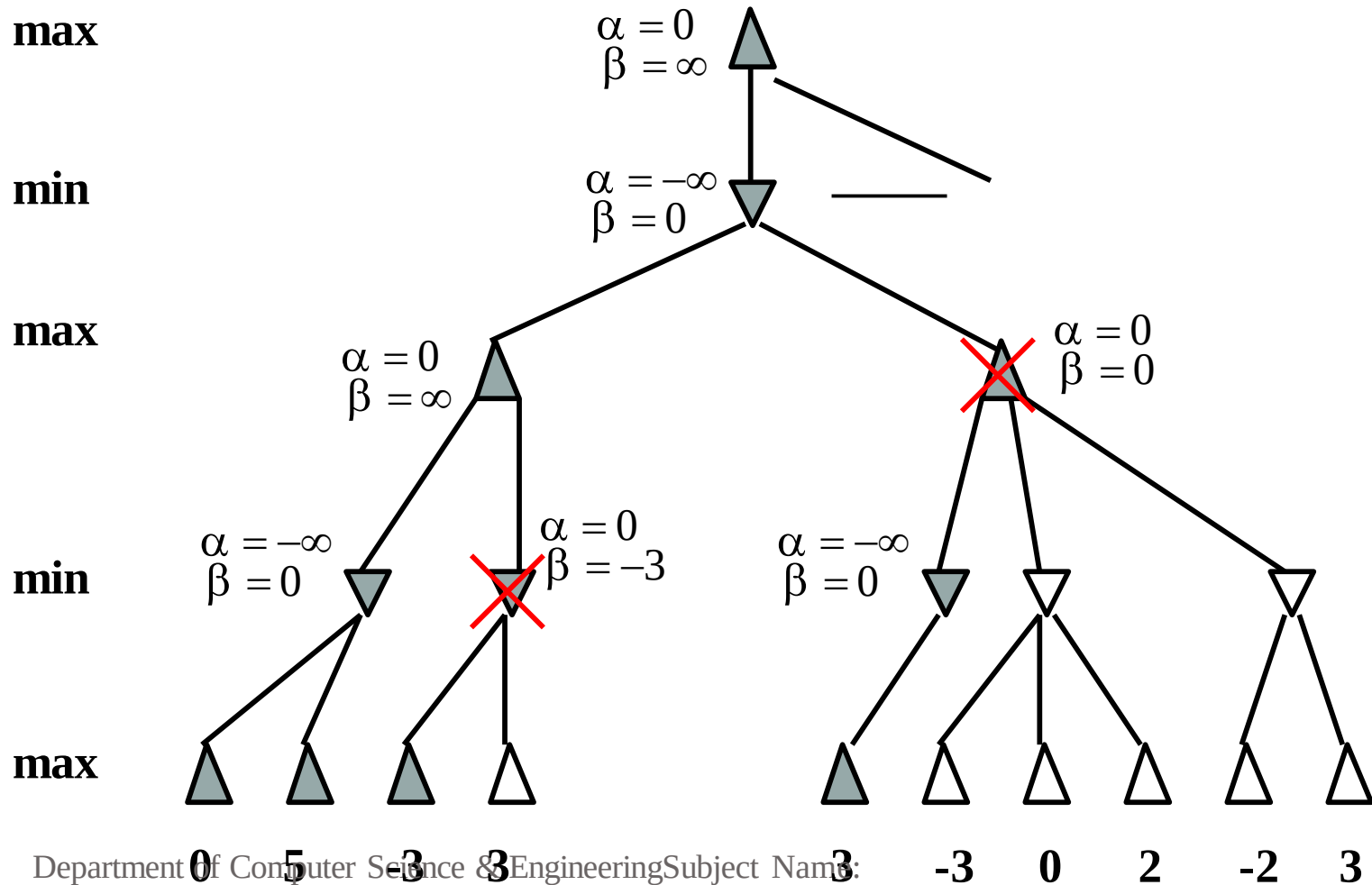


α - β pruning example





Final tree





Properties of α - β

- Pruning **does not** affect final result
- Good move ordering improves effectiveness of pruning
- With "perfect ordering," time complexity = $O(b^{m/2})$
→ **doubles** depth of search
- A simple example of the value of reasoning about which computations are relevant (a form of **metareasoning**)





Why is it called α - β ?

- α is the value of the best (i.e., highest-value) choice found so far at any choice point along the path for *max*
- If v is worse than α , *max* will avoid it
→ prune that branch
- Define β similarly for *min*

MAX

MIN

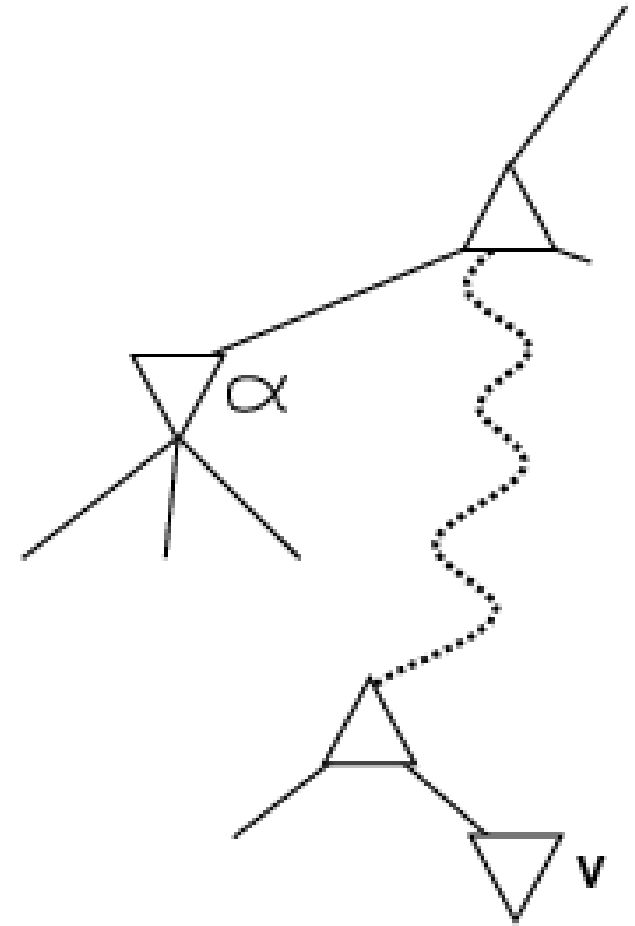
..

..

..

MAX

MIN





Deterministic games in practice

- Checkers: Chinook ended 40-year-reign of human world champion Marion Tinsley in 1994. Used a precomputed endgame database defining perfect play for all positions involving 8 or fewer pieces on the board, a total of 444 billion positions.
- Chess: Deep Blue defeated human world champion Garry Kasparov in a six-game match in 1997. Deep Blue searches 200 million positions per second, uses very sophisticated evaluation, and undisclosed methods for extending some lines of search up to 40 ply.
- Othello: human champions refuse to compete against computers, who are too good.
- Go: human champions refuse to compete against computers, who are too bad. In go, $b > 300$, so most programs use pattern knowledge bases to suggest plausible moves.





Summary

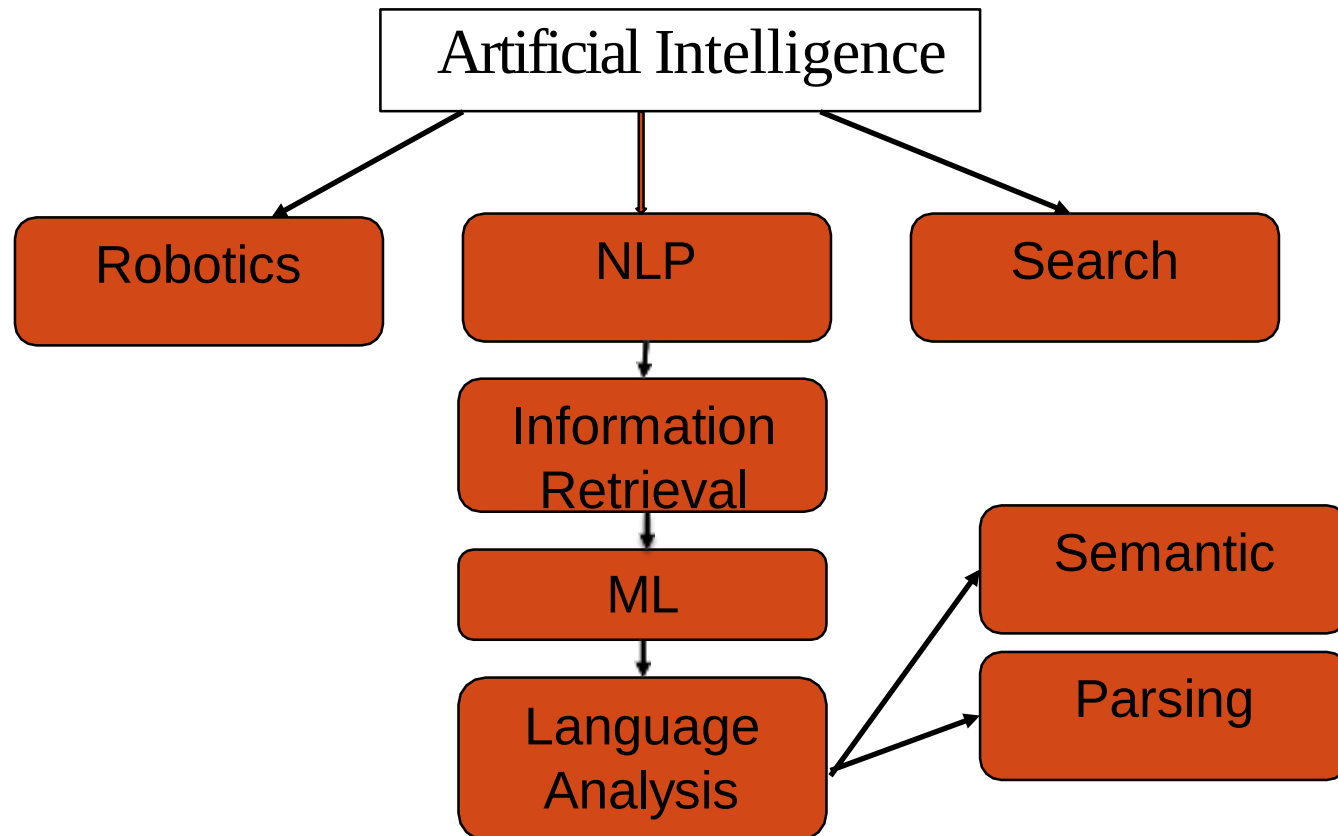
- Games are fun to work on!
- They illustrate several important points about AI
- perfection is unattainable → must approximate
- good idea to think about what to think about

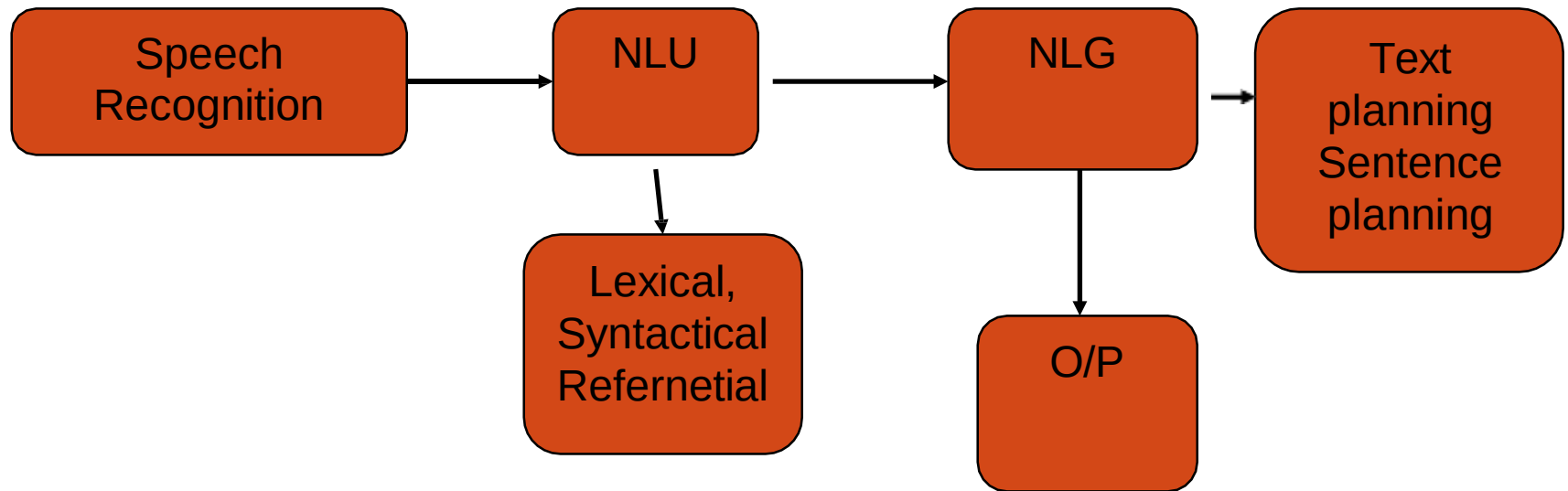




Natural Language Processing (NLP)









Communication Typical communication episode

S (speaker) wants to convey P (proposition) to H (hearer) using W (words in a formal or natural language)

1. Speaker

- **Intention:** S wants H to believe P
- **Generation:** S chooses words W
- **Synthesis:** S utters words W

2. Hearer

- **Perception:** H perceives words W'' (ideally $W'' = W$)
- **Analysis:** H infers possible meanings $P_1, P_2, \dots, P_n$ for W''
- **Disambiguation:** H infers that S intended to convey P_i (ideally $P_i = P$)
- **Incorporation:** H decides to believe or disbelieve P_i



Natural Language Processing (NLP)

1. Natural Language Understanding

- Taking some spoken/typed sentence and working out what it means

2. Natural Language Generation

- Taking some formal representation of what you want to say and working out a way to express it in a natural (human) language (e.g., English)





Why NLP??

- Huge amount of data on Internet
- Classify text into categories
- Index and search large texts
- Automatic translation
- Speech understanding
- Information Extraction
- Automatic summarization
- Knowledge Acquisition
- Text generation /dialogs





Applications of Nat. Lang. Processing

- Machine Translation
- Database Access
- Information Retrieval
 - Selecting from a set of documents the ones that are relevant to a query
- Text Categorization
 - Sorting text into fixed topic categories
- Extracting data from text
 - Converting unstructured text into structure data
- Spoken language control systems
- Spelling and grammar checkers





Contd..

- Speech recognition
- Sentiment analysis
- Machine translation
- Chat bots





Natural language understanding

Raw speech signal

● ↓ **Speech recognition**

Sequence of words spoken

● ↓ **Syntactic analysis** using knowledge of the grammar

Structure of the sentence

● ↓ **Semantic analysis** using info. about meaning of words

Partial representation of meaning of sentence

● ↓ **Pragmatic analysis** using info. about context

Final representation of meaning of sentence





Natural Language Understanding

● Input/Output data

Processing stage

Other data used

Frequency spectrogram



Word sequence

“He loves Mary”

/entence structure

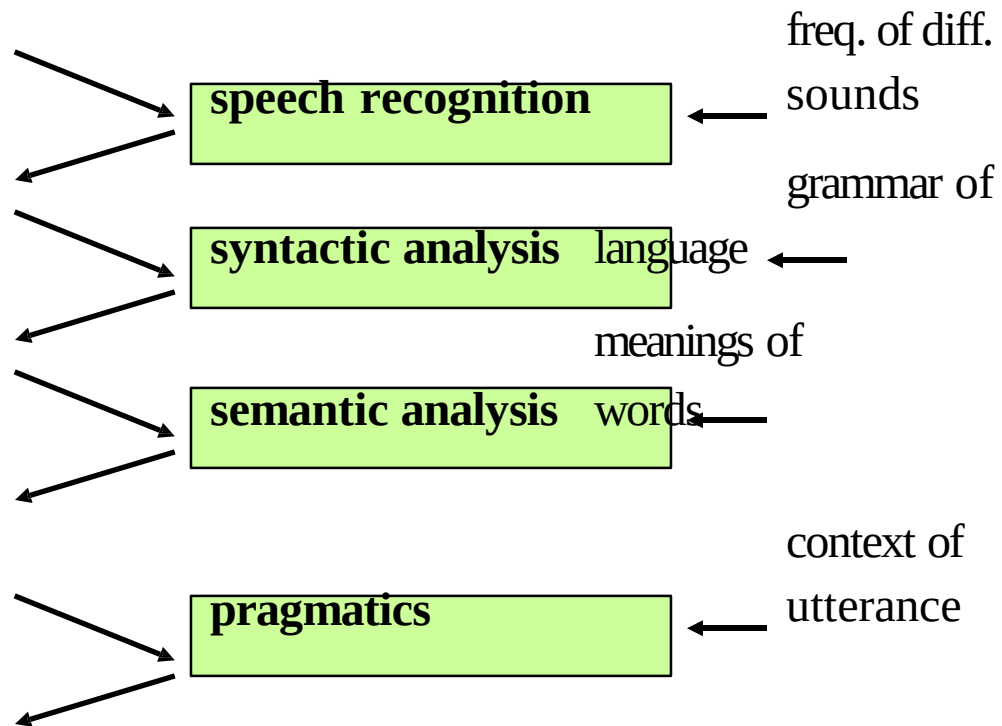
He loves Mary

Partial Meaning

$\exists x \text{ loves}(x, \text{mary})$

/entence meaning

$\text{loves}(\text{john}, \text{mary})$



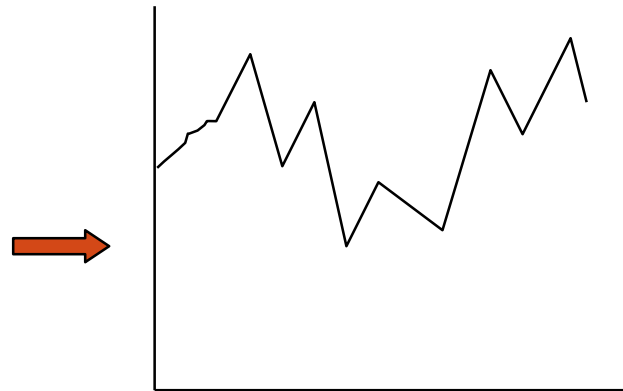


Speech Recognition (1 of 3)

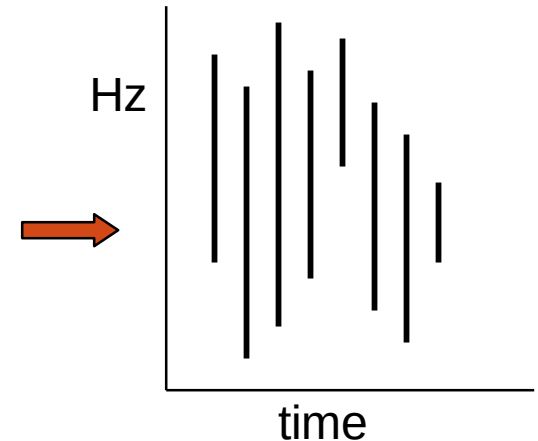
Input
(microphone records voice)



Analog Signal



Freq. spectrogram
(e.g. Fourier transform)





Speech Recognition (2 of 3)

● Frequency spectrogram

● Basic sounds in the signal (40-50 **phonemes**) (e.g. “a” in “cat”)

- Template matching against db of phonemes
 - Using dynamic time warping (speech speed)

● Constructing words from phonemes (e.g. “th”+”i”+”ng”=thing)

- Unreliable/probabilistic phonemes (e.g. “th” 50%, “f” 30%, ...)
- Non-unique pronunciations (e.g. tomato),
- statistics of transitions phonemes/words (hidden Markov models)

● Words



Speech Recognition - Complications

- No simple mapping between sounds and words
 - Variance in pronunciation due to gender, dialect, ...
 - Restriction to handle just one speaker
 - Same sound corresponding to diff. words
 - e.g. bear, bare
 - Finding gaps between words
 - “how to recognize speech”
 - “how to wreck a nice beach”
 - Noise





Linguistic level of Analysis

- Speech
- Written Language
- Phonology/sounds/letters
- Morphology
- Syntax
- Semantics
- Interaction between levels





Issues in Syntax

- "the dog ate my homework"
- POS tagging
 - Verb, adjective, adverb, noun, pronoun
- Identify collocation
 - Mother in law
 - Hot dog
- Anaphora Resolution
 - The dog entered into my room and it scared me.





Syntactic Analysis

- Rules of syntax (grammar) specify the possible organization of words in sentences and allows us to determine sentence's structure(s)
 - “John saw Mary with a telescope”
 - John saw (Mary with a telescope)
 - John (saw Mary with a telescope)
- Parsing: given a sentence and a grammar
 - Checks that the sentence is correct according with the grammar and if so returns a **parse tree** representing the structure of the sentence





Syntactic Analysis - Grammar

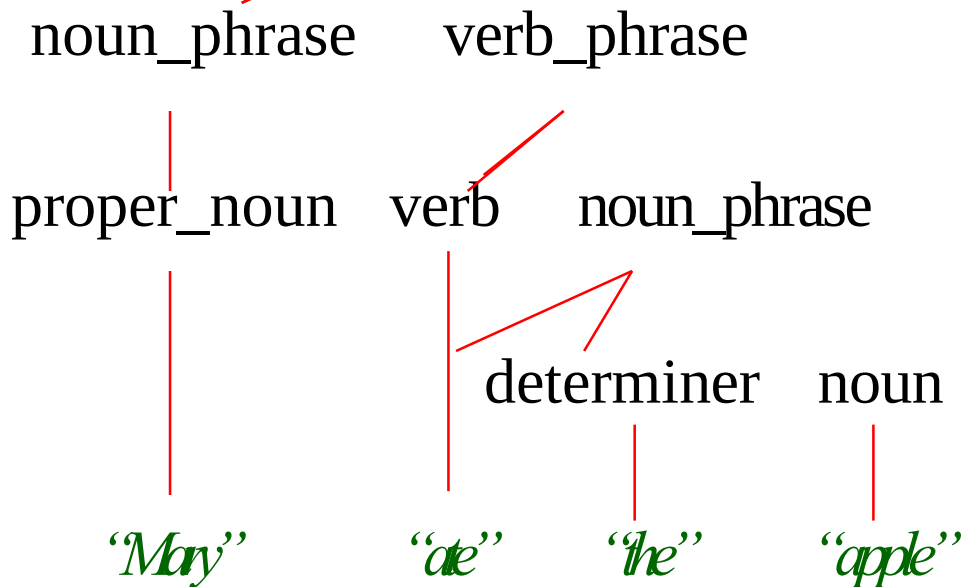
- sentence $\rightarrow$ noun_phrase, verb_phrase
- noun_phrase $\rightarrow$ proper_noun
- noun_phrase $\rightarrow$ determiner, noun
- verb_phrase $\rightarrow$ verb, noun_phrase
- proper_noun $\rightarrow$ [mary]
- noun $\rightarrow$ [apple]
- verb $\rightarrow$ [ate]
- determiner $\rightarrow$ [the]





Syntactic Analysis - Parsing

sentence





Syntactic Analysis – Complications (1)

- Number (singular vs. plural) and gender
 - **Sentence** → **noun_phrase(n), verb_phrase(n)**
 - **proper_noun(s)** → [mary]
 - **noun(p)** → [apples]
- Adjective
 - **noun_phrase** → **determiner, adjectives, noun**
 - **Adjectives** → adjective, adjectives
 - **Adjective** → [ferocious]
- Adverbs ...





Syntactic Analysis – Complications (2)

- Handling ambiguity
 - Syntactic ambiguity: “fruit flies like a banana”
- Having to parse syntactically incorrect sentences





Issues in Semantics

- Understand Language

- Plant-industrial plant, Plant-living organism

Eg: There are producing about 10,000 automobiles in the plant.

The sea flora consist of 1000 different plant species.

- Bank-where we keep money, bank-river bank

- Words are ambiguous

- Why semantics important

- Wrong machine translation

- Wrong information retrieval

- Wrong referents-anaphora resolution





Semantic Analysis

- Generates (partial) meaning/representation of the sentence from its syntactic structure(s)
- Compositional semantics: meaning of the sentence from the meaning of its parts:
 - Sentence: A tall man likes Mary
 - Representation: $\exists x \text{ man}(x) \ \& \ \text{tall}(x) \ \& \ \text{likes}(x, \text{mary})$
- Grammar + Semantics
 - **Sentence (Smeaning) → noun_phrase (NPmeaning), verb_phrase (VPmeaning), combine (NPmeaning, VPmeaning, Smeaning)**



Semantic Analysis – Complications

- Handling ambiguity
 - Semantic ambiguity: “I saw the prudential building flying into Boston”





Pragmatics

- Uses context of utterance
 - Where, by who, to whom, why, when it was said
 - Intentions: *inform, request, promise, criticize, ...*
- Handling Pronouns
 - ◄ ● “Mary eats apples. She likes them.”
 - She=“Mary”, them=“apples”.
- Handling ambiguity
 - Pragmatic ambiguity: “**you’re late**”: What’s the speaker’s intention: informing or criticizing?





Natural Language Generation

- Talking back! 😊
- What to say or text planning
 - flight(AA,london,boston,\$560,2pm),
 - flight(BA,london,boston,\$640,10am),
- How to say it
 - “There are two flights from London to Boston. The first one is with American Airlines, leaves at 2 pm, and costs \$560 ...”
- Speech synthesis
 - Simple: Human recordings of basic templates
 - More complex: string together phonemes in phonetic spelling of each word
 - Difficult due to stress, intonation, timing, liaisons between words



Expert Systems





What is Expert System?

- An expert system, is an interactive computer-based decision tool that uses both facts and heuristics to solve difficult decision making problems, based on knowledge acquired from an expert.
- Inference engine + Knowledge = Expert system
(Algorithm + Data structures = Program in traditional computer)
- First expert system, called DENDRAL, was developed in the early 70's at Stanford University.





Introduction

Expert systems are computer applications which embody some non-algorithmic expertise for solving certain types of problems. For example :

- Diagnostic applications
- Play chess
- Make financial planning decisions
- Configure computers
- Monitor real time systems
- Underwrite insurance policies
- Perform many services which previously required human expertise.





Expert System Shells

- Many expert systems are built with products called expert system shells. A shell is a piece of software which contains the user interface, a format for declarative knowledge in the knowledge base, and an inference engine.
- The knowledge and system engineers use these shells in making expert systems.





- **Knowledge engineer** : uses the shell to build a system for a particular problem domain.
- **System engineer** : builds the user interface, designs the declarative format of the knowledge base, and implements the inference engine.

Depending on the size of the system, the knowledge engineer and the system engineer might be the same person.





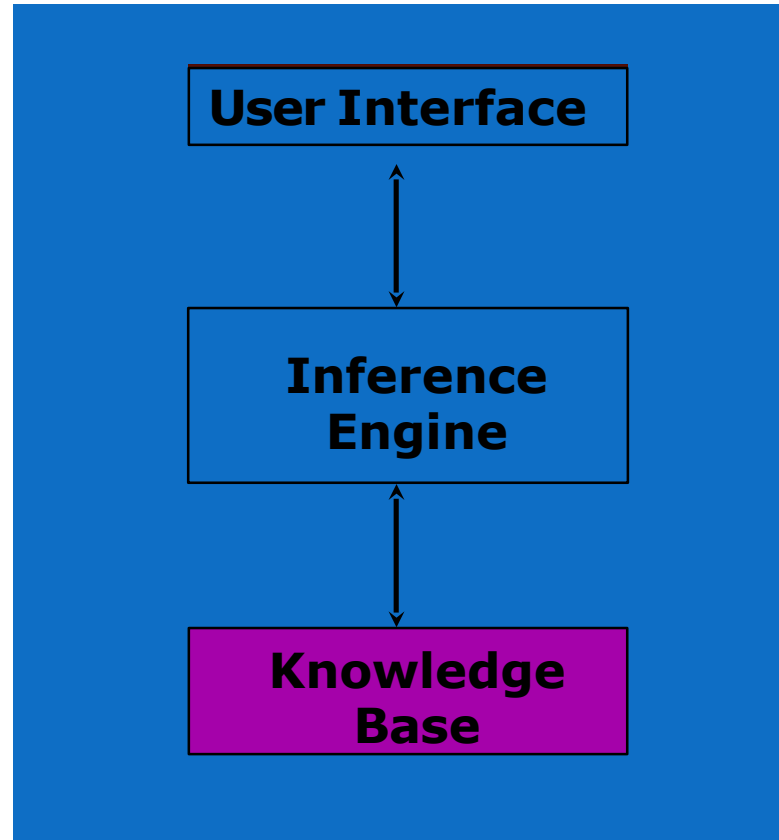
Human Expert Behaviors

- Recognize and formulate the problem
- Solve problems quickly and properly
- Explain the solution
- Learn from experience
- Restructure knowledge
- Break rules
- Determine relevance
- Degrade gracefully





Expert System Components





The knowledge base contains the knowledge necessary for understanding, formulating, and solving problems.

Components of Knowledge Base

The knowledge base of an ES is a store of both, factual and heuristic knowledge.

- Factual Knowledge – It is the information widely accepted by the Knowledge Engineers and scholars in the task domain.
- Heuristic Knowledge – It is about practice, accurate judgment, one's ability of evaluation, and guessing.
- The Inference Engine, *brain* of the ES.
- The control structure (rule interpreter)
- Provides methodology for reasoning





Expert System Architecture

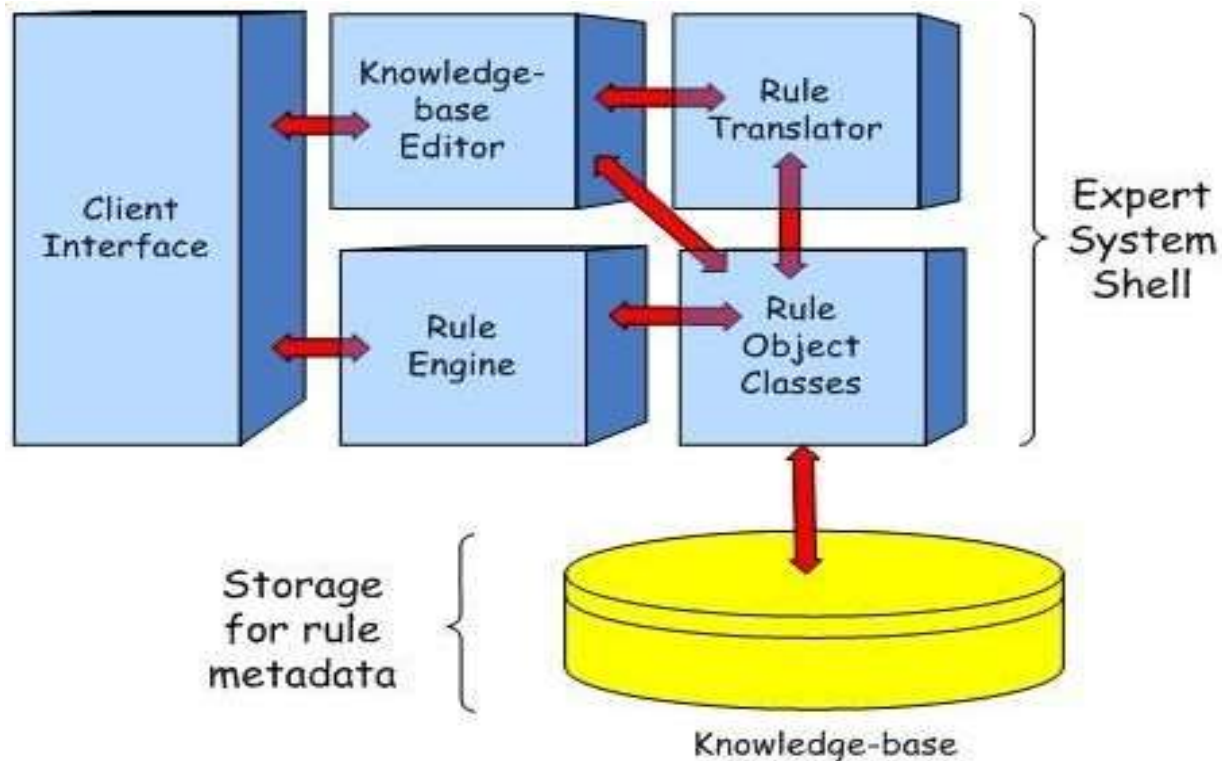


Figure 2 - Expert System Architecture



- **The Client Interface** processes requests for service from system-users and from application layer components.
- **The Knowledge-base Editor** is a simple editor that enable a subject matter expert to compose and add rules to the Knowledge-base.
- **Rule Translator** converts rules from one form to another i.e; their original form to a machine-readable form.
- **The Rule Engine(*inference engine*)** is responsible for executing Knowledge-base rules.
- **The shell** component, Rule Object Classes, is a container for object classes supporting.





The Knowledge Base

- **Inferential knowledge** : stored in decision rules
 - If Premise then action(Certainty Factor CF)
 - If A & B then C(0 .6)
 - If CF represents the inferential certainty
- **Static Knowledge** :
 - Natural Language Dictionary
 - Lists(Sterile sites)
 - Tables(gram stain, morphology,aerobicity)
- **Dynamic knowledge** : stored in the context free
 - Patient specific
 - hierarchical structure : Patient Cultures, organisms,
 - <object, Attribute, value>,triples :<Org1,Identity, Strep>
 - A CF is used for factual certainty<Org1,Identity,Staph,o.6>



Components and Interfaces

- **User interface** : The code that controls the dialog between the user and the system.
- **Knowledge base** : A declarative representation of the expertise often in IF THEN rules .
- **Inference engine** : The code at the core of the system which derives recommendations from the knowledge base and problem specific data in working storage.
- **Working storage** : The data which is specific to a problem being solved.





Roles of Individuals who interact

With the system

- **Domain expert** : The individuals who currently are experts in solving the problems; here the system is intended to solve.
- **Knowledge engineer** : The individual who encodes the expert's knowledge in declarative form that can be used by the expert system.
- **User** : The individual who will be consulting with the system to get advice which would have been provided by the expert.
- **System engineer** : builds the user interface, designs the declarative format of the knowledge base, and implements the inference engine.





Expert System Benefits

- **Increased Output and Productivity**
- **Decreased Decision Making Time**
- **Increased Process and Product Quality**
- **Reduced Downtime**
- **Capture Scarce Expertise**
- **Flexibility**
- **Easier Equipment Operation**
- **Elimination of Expensive Equipment.**





Capabilities of Expert Systems





☐ Fuzzy logic

- A specialty research area in computer science that allows shades of gray and does not require everything to be simply yes/no, or true/false

☐ Backward chaining

- A method of reasoning that starts with conclusions and works backward to the supporting facts

☐ Forward chaining

- A method of reasoning that starts with the facts and works forward to the conclusions

Schematic





Rules for a Credit Application

Mortgage application for a loan for \$100,000 to \$200,000

If there are no previous credits problems, and

If month net income is greater than 4x monthly loan payment, and

If down payment is 15% of total value of property, and

If net income of borrower is $> \$25,000$, and

If employment is > 3 years at same company

Then accept the applications

Else check other credit rules



Expert Systems Development



Domain

- The area of knowledge addressed by the expert system.



Expert System Limitations

- Knowledge is not always readily available
- Expertise can be hard to extract from humans
- Each expert's approach may be different, yet correct
- Hard, even for a highly skilled expert, to work under time pressure
- Expert system users have natural cognitive limits
- ES work well only in a *narrow domain* of knowledge.





- ? Most experts have no independent means to validate their conclusions
- ? Experts' vocabulary often limited and highly technical
- ? Knowledge engineers are rare and expensive
- ? Lack of trust by end-users
- ? Knowledge transfer subject to a host of perceptual and judgmental biases
- ? ES may not be able to arrive at valid conclusions
- ? ES sometimes produce incorrect recommendations





MYCIN

- MYCIN was developed at Stanford by Shortliffe (1970's). It was an expert system for diagnosing blood diseases. It was a precursor to today's expert systems and acts as an ideal case study.
- MYCIN is a rule based ES using backward chaining.
- A typical rule
- IF stain of organism is gram negative
- AND morphology is rod
- AND is anaerobic
- THEN suggestive that class is enterobacteriaceae (CF 0.8)
- Inexact reasoning was employed using certainty factors. T
- number on the scale
- -1 to 1. -1 being definitely false +1 definitely true.





- To make the system acceptable to users (doctors) MYCIN incorporated
- **Natural Language** Users entered information in English and were answered in English.
- **Spelling** A spelling checker made intelligent corrections to the input.
- **Explanation** Both WHY, WHY NOT and HOW explanation facilities were provided.
- **Alternative Recommendations** Several suggestions were often offered listed with some priority, allowing the doctor some freedom of selection.
- MYCIN has two phases in its approach,
- **a diagnosis and a prescription phase.**
- In the first phase the nature of the infection and the organism causing the infection are determined (hopefully).
- The prescription phase then indicates the drugs for the treatment taking into account any side effects they may have on the patient.
- MYCIN consists of about 500 rules. It is a backward chaining rule based system using a depth-first search strategy.



- COMMENT: MYCIN begins the session by asking for general information about the patient.
- STEPS
- MYCIN: What is the patient's name?
- User: Alex
- MYCIN: What is the patient's age?
- User: 45
- MYCIN: What is the patient's sex?
- User: MALE
- MYCIN: What is the patient's race?
- User: CAUCASIAN
- COMMENT: After obtaining background information, MYCIN attempts to determine the nature of the infection. It first seeks help from the user to see if this information is known from test results that produced positive cultures.



- The major steps in designing a backward chaining ES are:
- 1. define the problem;
- 2. define the goals;
- 3. define the goal rules;
- 4. expand/refine the system;
- 5. design the interface;
- 6. evaluate the system

These tasks are iterated until a final system is produced.





Uncertain Modifier	CF
Definitely not	-1
Almost certainly not	-0.8
Probably not	-0.6
Maybe not	-0.4
Unknown	-.2 to .2
Maybe	0.4
Probably	0.6
Almost certainly	0.8
Definitely	1.0



Example:

- It is probably glandular fever
- then we can think of this as
- It is glandular fever CF 0.6 and
- It is almost certainly not measles then we can think of this as
- It is measles CF -0.8





Example of a Decision Rule

RULE – 507

IF:

- The infection which requires therapy is meningitis.
- Organisms were not seen on the stain of the culture.
- The type of the infection is bacterial.
- The age of the patient is between 15 and 55 years.

Then:

- The organisms that might be causing the infection and diplococcus – pneumoniae and neisseria – meningitidis.





Dendral Expert System





Content..

- What is an Expert System?
- What is Dendral?
- Why it was needed?
- History
- Heuristic
- Parts of Dendral
 - Heuristic Dendral
 - Meta Dendral
- Plan Generate test paradigm
- Knowledge Engineering
- Derived Systems





What is an Expert System

- In artificial intelligence, an expert system is a computer system that emulates the decision making ability of a human expert.
- The first expert systems were created in the 1970s and then proliferated in the 1980s.





What is Dendral

- **Dendral** was an influential pioneer project in artificial intelligence (AI) of the 1960s, and the computer software expert system that it produced.
- The name Dendral is a portmanteau of the term "Dendritic Algorithm"
- It was written in Lisp (programming language), which was considered the language of AI because of its flexibility





Why it was needed

- Its primary aim was to help organic chemists in identifying unknown organic molecules, by analyzing their **mass spectra** and using knowledge of chemistry

Mass Spectrum:

- A mass spectrum is a plot of the ion signal as a function of the mass to charge ratio.





Inventors

- It was done at Stanford University by Edward Feigenbaum, Bruce G. Buchanan, Joshua Lederberg, and Carl Djerassi. It began in 1965 and spans approximately half the history of AI research





History..

- During the 20th century, the question “Can Machine think??” became popular among scientists.
Also around mid 20th century, science, especially biology, faced a fast increasing need to develop a "man computer symbiosis", to aid scientists in solving problems.
- In the early 1960s, Joshua Lederberg started working with computers and quickly became tremendously interested in creating interactive computers to help him in his exobiology research. Specifically, he was interested in designing computer systems that to help him study alien organic compound.





contd..

- Feigenbaum was an expert in programming languages and heuristics, and helped Lederberg design a system that replicated the way Carl Djerassi solved structure elucidation problems.
- They devised a system called Dendritic Algorithm (Dendral) that was able to generate possible chemical structures corresponding to the mass spectrometry data as an output
- The Dendral team recruited Bruce Buchanan to extend the Lisp program initially written by Georgia Sutherland.





Heuristics

- The use of heuristics to solve problems is called "heuristics programming", and was used in Dendral to allow it to replicate in machines the process through which human experts induce the solution to problems via rules of thumb and specific information.
- Heuristics programming was a major approach and a giant step forward in artificial intelligence, as it allowed scientists to finally automate certain traits of human intelligence.
- As Herbert A. Simon said in *The Sciences of the Artificial*, "if you take a heuristic conclusion as certain, you may be fooled and disappointed; but if you neglect heuristic conclusions altogether you will make no progress at all."





Parts of Dendral

- Heuristic Dendral
- Meta Dendral





Heuristic Dendral

- Heuristic Dendral is a program that uses mass spectra or other experimental data together with knowledge base of chemistry, to produce a set of possible chemical structures that may be responsible for producing the data.
- A mass spectrum of a compound is produced by a mass spectrometer, and is used to determine its molecular weight, the sum of the masses of its atomic constituents.





CONTD..

- The PRELIMINARY INFERENCE MAKER
- The DATAADJUSTOR
- The STRUCTURE GENERATOR
- The PREDICTOR
- EVALUATION FUNCTION



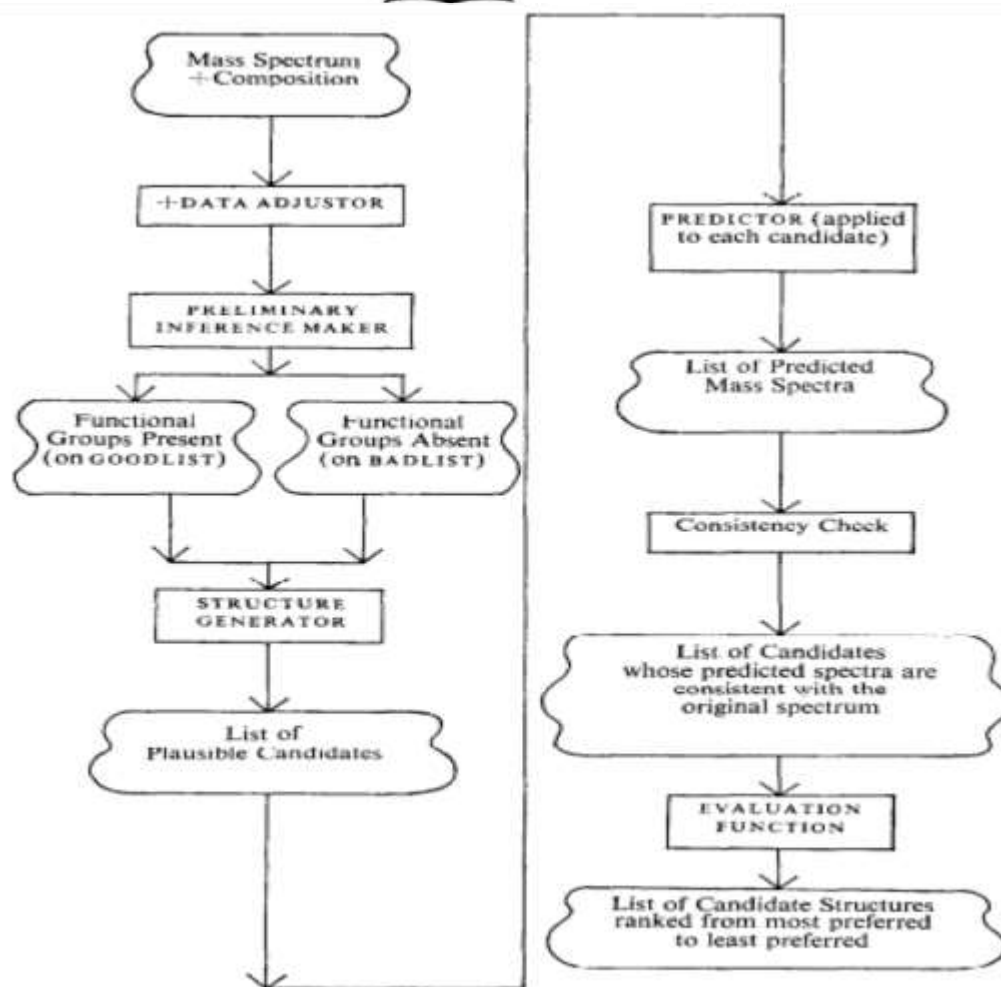


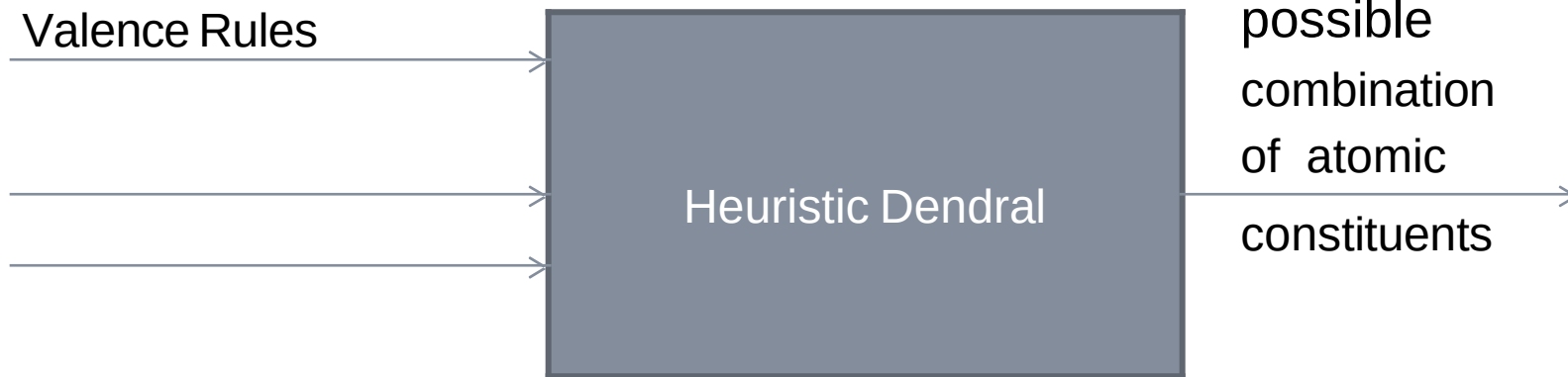
Figure 2. General design of HEURISTIC DENDRAL



For Example....

- Consider Water molecule (H_2O)
H=1.01 Molecular Weight=18
O=8.00 Mass Spectrum=18 units

input mass knowledge of
atomic mass number
Valence Rules





Heuristic Dendral(contd..)

- New graph theoretic algorithms were invented by Lederberg, Harold Brown, and others that generate all graphs with a specified set of nodes and connection types (chemical atoms and bonds) with or without cycles.
- Moreover, the team was able to prove mathematically that the generator is complete, in that it produces all graphs with the specified nodes and edges, and that it is non redundant in that the output contains no equivalent graphs (e.g., mirror images).





Meta Dendral

- Meta Dendral is a machine learning system that receives the set of possible chemical structures and corresponding mass spectra as input, and proposes a set of rules of mass spectrometry that correlate structural features with processes that produce the mass spectrum.
- These rules would be fed back to Heuristic Dendral (in the planning and testing programs described below) to test their applicability





Plan generate test paradigm

- The plan generate test paradigm is the basic organization of the problem solving method, and is a common paradigm used by both Heuristic Dendral and Meta Dendral systems.
- The generator (later named CONGEN) generates potential solutions for a particular problem, which are then expressed as chemical graphs in Dendral. However, this is feasible only when the number of candidate solutions is minimal. When there are large numbers of possible solutions, Dendral has to find a way to put constraints that rules out large sets of candidate solutions.





Knowledge Engineering

- The primary aim of knowledge engineering is to attain a productive interaction between the available knowledge base and problem solving techniques.
- This is possible through development of a procedure in which large amounts of task specific information is encoded into heuristic programs. Thus, the first essential component of knowledge engineering is a large “knowledge base.”
- This “knowledge base” is used both to search for possible chemical structures that match the input data, and to learn new “general rules” that help prune searches.





Derived Systems

Many System were derived from Dendral including

- MYCIN
- MOLGEN,
- MACSYMA,
- PROSPECTOR
- XCON
- STEAMER





Speech Recognition Systems





INTRODUCTION

- Speech recognition is a process by which a computer takes a speech signal (recorded using a microphone) and converts it into words in real-time. It is achieved by following certain steps and the software responsible for it is known as a '**Speech Recognition System**'
- SR systems are usually implemented in the form of dictation software and intelligent assistants in personal computers, smartphones, web browsers and many other devices.





CHALLENGES IN THE DESIGN OF A SR SYSTEM

SR systems have to deal with a large number of challenges like :-

- The speaker's voice is often accompanied by **surrounding noise** which makes their accurate recognition difficult.
- A speaker may speak a **number of different words** and all of these words have to be accurately recognized.
- **Accent** of speaking varies from person to person and this is a very big challenge
- A speaker may **speak** something very **quickly** and all of the words spoken have to be individually recognized accurately.





TYPES OF SR SYSTEMS

- **Speaker Dependent SR systems** : Work by learning the unique characteristics of a single person's voice and depend on the speaker for training.
- **Speaker Independent SR systems** : Designed to recognize anyone's voice, so no training is involved.





BASIC PRINCIPLES OF SPEECH RECOGNITION

- The smallest unit of spoken language is known as a **Phoneme**.
- The English language contains approximately 44 phonemes representing all the vowels and consonants that we use for speech.
- We can take the example of a typical word such as **moon** which can be broken down into three phonemes: **m, ue, n**.





- To interpret speech we must have a way of identifying the components of spoken words and **phonemes** act as identifying markers within speech.
- An algorithm has to be used to interpret the speech further. The **Hidden Markov Model** is a commonly used mathematical model used to do this.
- To create a speech recognition engine, **a large database of models is created** to match each phoneme.
- When a comparison is performed, the **most likely match is determined** between the spoken phoneme and the stored one, and further computations are performed.





COMPONENTS OF SPEECH RECOGNITION

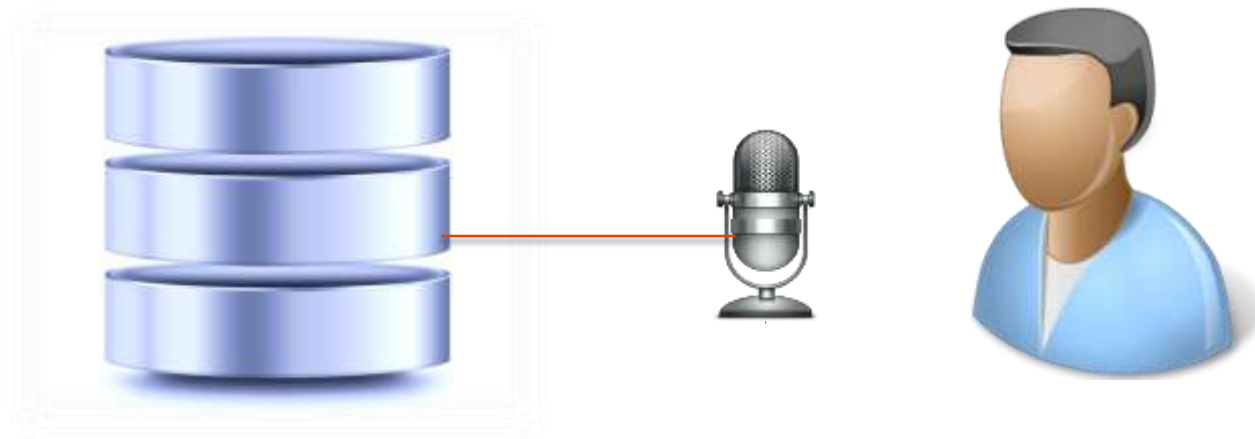
- **Corpus Collection :**

Database consisting of speech data that built from multiple speech samples.



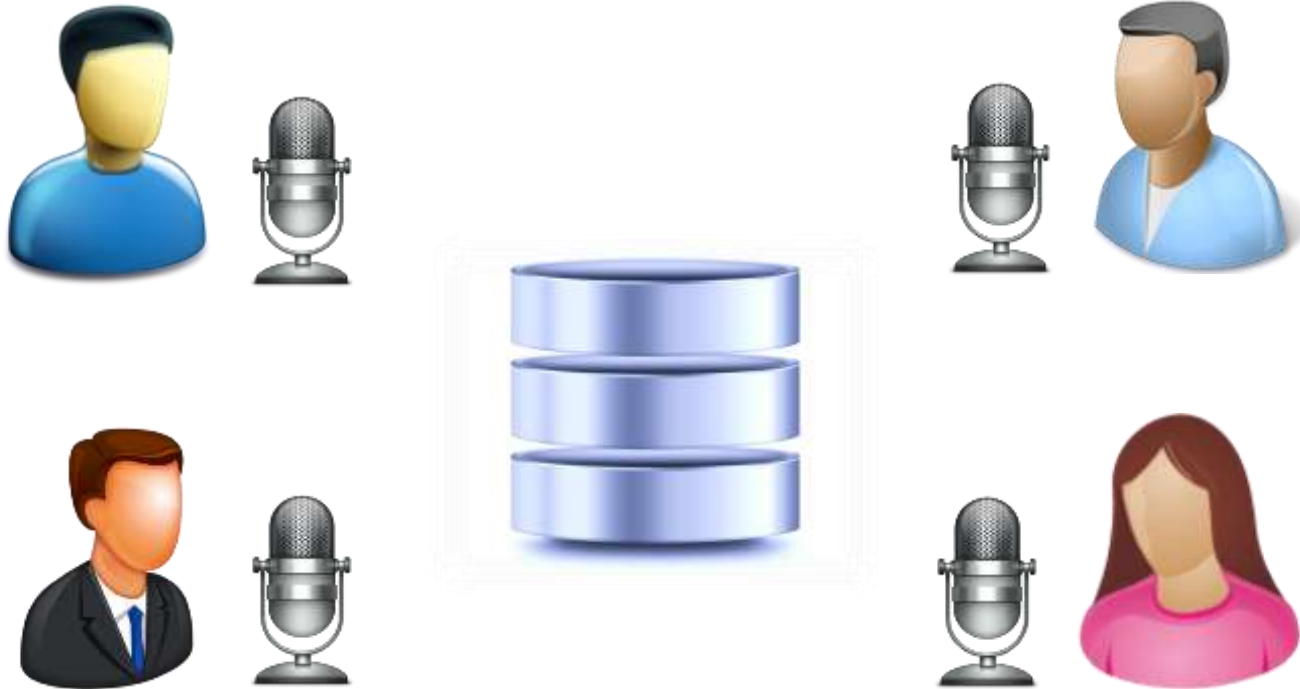


- Corpus collection construction for a **speaker-dependent SR system** :-





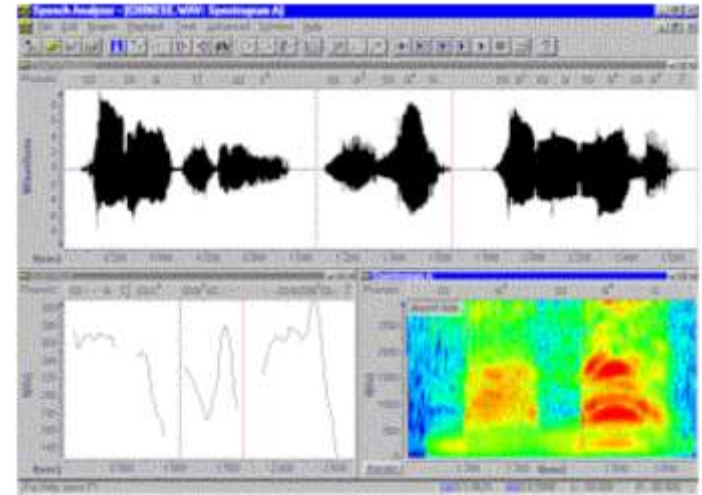
- Corpus collection construction for a **speaker-independent** SR system.





- **Signal Analyzer :**

Analyses the speech signal and removes the background noise thus focusing only on the speaker's speech .



- **Acoustic Model :**

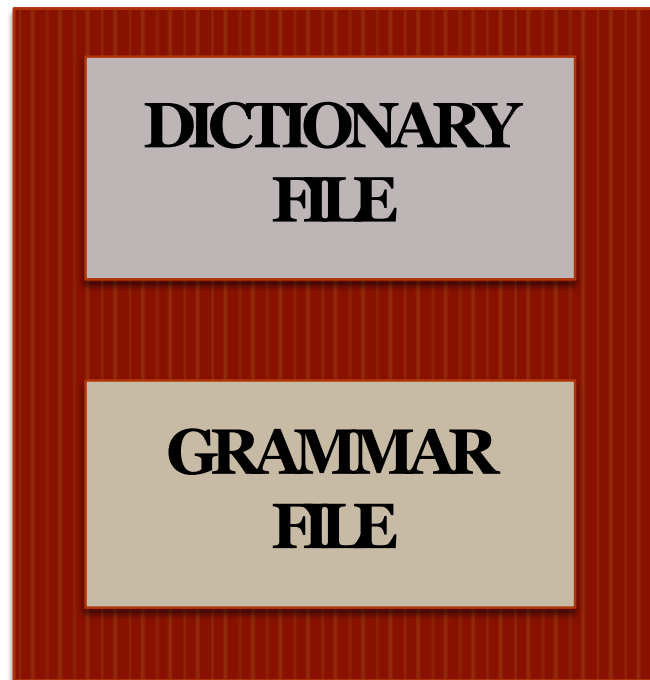
Identifies phonemes from the speech sample using a probability based mathematical model.

ACOUSTIC MODEL





- **Language Model** : Identifies words and thus sentences uttered by the speaker from the phonemes by **making use of a dictionary file and grammar file.**





PROCESS OF SPEECH RECOGNITION





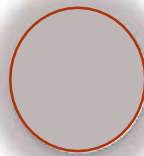
SPEECH ANALYZER



/p/-/ae/-/n/



ACOUSTIC MODEL



CORRECT

/p/--/ae/--/n/



/p/--/ae/--/n/



**TRAINED HIDDEN
MARKOV MODEL**



LANGUAGE MODEL

DICTIONARY FILE

GRAMMAR FILE

/p/-/ae/-/n/

pain

pain

pain

TEXT OUTPUT



Factors affecting the accuracy of a SR system :-

- Vocabulary size and confusability
- Speaker dependence vs. independence
- Isolated, discontinuous, or continuous speech
- Task and language constraints
- Read vs. spontaneous speech
- Adverse conditions





APPLICATIONS

- Health Care
- Military - High Performance Aircrafts
 - Air Traffic Control Systems
- Telephony - Smart-phones
 - Customer Helpline Services
- Personal Computers





SIRI AND GOOGLE NOW



**Intelligent Personal Assistant
developed by Apple.**



**Google Now is an intelligent personal
assistant developed by Google.**

**Both use a combination of speaker- dependent and
speaker-independent sr systems**

